

The allegory axioms, and what defines them in the Frobenius calculus

§1. Notation

Traditionally Adjunction $F \dashv G$ is defined as $F(x) \leq y$ iff $x \leq G(y)$, and you derive laws like this:

$$\begin{aligned} F(x) \leq F(x) &\implies x \leq G(F(x)) \implies F(x) \leq F(G(F(x))) \\ G(y) \leq G(y) &\implies F(G(y)) \leq y \implies G(F(G(y))) \leq G(y) \end{aligned}$$

By define $X : * \mapsto x$, $Y : * \mapsto y$ and use diagram order, application can be replaced by composition: $XF \leq Y$ iff $X \leq YG$

- write F as $/$, G as $\backslash$

- render $a \leq b$ as $\boxed{\begin{smallmatrix} a \\ b \end{smallmatrix}}$

- **rewrite** $X \leq XF$ as $1 \leq F$

(1a)

$$\boxed{\frac{X/}{Y}} = \boxed{\frac{X}{Y\backslash}}$$

adj

$$\begin{aligned} \boxed{\frac{X/}{X/}} &\xRightarrow{\text{adj}} \boxed{\frac{X}{X\backslash}} \xRightarrow{\text{rewrite}} \boxed{\frac{*}{\bigwedge}} \xRightarrow[\text{unit}]{\text{put } \backslash \text{ on left}} \boxed{\frac{\backslash}{\bigwedge}} \\ &\xRightarrow[\text{put } / \text{ on right}]{\text{put } / \text{ on right}} \boxed{\frac{/}{\bigvee}} \\ \boxed{\frac{Y\backslash}{Y\backslash}} &\xRightarrow{\text{adj}} \boxed{\frac{Y\bigvee}{Y}} \xRightarrow{\text{rewrite}} \boxed{\frac{\bigvee}{*}} \xRightarrow[\text{counit}]{\text{put } / \text{ on left}} \boxed{\frac{\bigvee}{/}} \\ &\xRightarrow[\text{put } \backslash \text{ on right}]{\text{put } \backslash \text{ on right}} \boxed{\frac{\bigwedge}{\backslash}} \end{aligned}$$

now we have $\bigvee = /$ and $\bigwedge = \backslash$, and you just discovered string diagrams.

§1.1. Adjunctions

(1.1a)

$F \dashv G$	F 's type	monad FG	$1 \in FG$	$GF \in 1$	F preserves u	G preserves n	$FGF=F$	$GFG=G$
$\triangleleft \dashv \triangleright$	$A \rightarrow A \otimes A$	$\triangleleft \triangleright = 1$	$1 \in \triangleleft \triangleright$	$\triangleright \triangleleft \in 1$	$(RuS) \triangleleft = R \triangleleft uS \triangleleft$	$(RnS) \triangleright = R \triangleright nS \triangleright$	$\triangleleft \triangleright \triangleleft = \triangleleft$	$\triangleright \triangleleft \triangleright = \triangleright$
$\multimap \dashv \multimap$	$A \rightarrow I$	$\multimap \multimap = T$	$1 \in \multimap \multimap$	$\multimap \multimap \in 1$	$(RuS) \multimap = R \multimap uS \multimap$	$(RnS) \multimap = R \multimap nS \multimap$	$\multimap \multimap \multimap = \multimap$	$\multimap \multimap \multimap = \multimap$
$\circ \dashv \circ$	$(A \rightarrow B) \rightarrow (B \rightarrow A)$	$\circ \circ = 1$	$R = R \circ \circ$	$R = R \circ \circ$	$(RuS) \circ = R \circ uS \circ$	$(RnS) \circ = R \circ nS \circ$	$R \circ \circ \circ = R \circ$	$R \circ \circ \circ = R \circ$
$\multimap \triangleleft \dashv \triangleright \multimap$	$(X \otimes A \rightarrow Y) \rightarrow (X \rightarrow Y \otimes A)$	1	$(\multimap \triangleleft \otimes 1) (1 \otimes \triangleright \multimap) = 1$	$(1 \otimes \multimap \triangleleft) (\triangleright \multimap \otimes 1) = 1$	$=$	$=$	$=$	$=$
$\triangleright \multimap \dashv \multimap \triangleleft$	$(X \rightarrow Y \otimes A) \rightarrow (X \otimes A \rightarrow Y)$	1	$(1 \otimes \multimap \triangleleft) (\triangleright \multimap \otimes 1) = 1$	$(\multimap \triangleleft \otimes 1) (1 \otimes \triangleright \multimap) = 1$	$=$	$=$	$=$	$=$
$\Delta \dashv \eta$	$(A \rightarrow B) \rightarrow (A \rightarrow B)^2$	$R \mapsto R \eta R = R$	$R \in R \eta R$	$R \eta S \in R$	$\Delta(RuS) = \Delta Ru \Delta S$	$(R \eta T) \eta (S \eta U) = (R \eta S) \eta (T \eta U)$	$R \eta R = R$	$R \eta R = R$
$u \dashv \Delta$	$(A \rightarrow B)^2 \rightarrow (A \rightarrow B)$	$(R, S) \mapsto (RuS, RuS)$	$R \in RuS$	$RuR \in R$	$(RuT) u (SuU) = (RuS) u (TuU)$	$\Delta(R \eta S) = \Delta R \eta \Delta S$	$RuR = R$	$RuR = R$
$\perp \dashv !$	$\{*\} \rightarrow (A \rightarrow B)$	—	—	$\perp \in R$	—	—	—	—
$\cdot S \dashv /S$	$(A \rightarrow B) \rightarrow (A \rightarrow C)$	S/S	$R \in (RS)/S$	$(T/S) S \in T$	$(RuT) S = RSuTS$	$(R \eta T)/S = R/S \eta T/S$	$((RS)/S) S = RS$	$((T/S) S)/S = T/S$
$S \cdot \dashv S \setminus$	$(B \rightarrow C) \rightarrow (A \rightarrow C)$	$S \setminus S$	$R \in S \setminus (SR)$	$S(S \setminus T) \in T$	$S(RuT) = SRuST$	$S \setminus (R \eta T) = S \setminus R \eta S \setminus T$	$S(S \setminus (SR)) = SR$	$S \setminus (S(S \setminus T)) = S \setminus T$
$R \eta \dashv R \Rightarrow$	$(A \rightarrow B) \rightarrow (A \rightarrow B)$	$X \mapsto R \Rightarrow (X \eta R)$	$X \in R \Rightarrow (X \eta R)$	$R \eta (R \Rightarrow Y) \in Y$	$R \eta (XuY) = (R \eta X) u (R \eta Y)$	$R \Rightarrow (X \eta Y) = (R \Rightarrow X) \eta (R \Rightarrow Y)$	$R \eta (R \Rightarrow (X \eta R)) = X \eta R$	$R \Rightarrow (R \eta (R \Rightarrow Y)) = R \Rightarrow Y$
$\mathcal{D} \dashv \cdot T$	$(A \rightarrow B) \rightarrow \text{Cor } A$	$R \mapsto (\mathcal{D}R) T$	$R \in (\mathcal{D}R) T$	$\mathcal{D}(AT) \in A$	$\mathcal{D}(RuS) = \mathcal{D}Ru \mathcal{D}S$	$(A \eta B) T = AT \eta BT$	$\mathcal{D}((\mathcal{D}R) T) = \mathcal{D}R$	$(\mathcal{D}(AT)) T = AT$
$\mathcal{R} \dashv T \cdot$	$(A \rightarrow B) \rightarrow \text{Cor } B$	$R \mapsto T(\mathcal{R}R)$	$R \in T(\mathcal{R}R)$	$\mathcal{R}(TA) \in A$	$\mathcal{R}(RuS) = \mathcal{R}Ru \mathcal{R}S$	$T(A \eta B) = T A \eta T B$	$\mathcal{R}(T(\mathcal{R}R)) = \mathcal{R}R$	$T(\mathcal{R}(TA)) = TA$
$\cdot f \dashv \cdot f^\circ$	$(A \rightarrow B) \rightarrow (A \rightarrow C)$	$f f^\circ$	$1 \in f f^\circ$	$f^\circ f \in 1$	$(RuS) f = R f u S f$	$(R \eta S) f^\circ = R f^\circ \eta S f^\circ$	$f f^\circ f = f$	$f^\circ f f^\circ = f^\circ$
$f^\circ \cdot \dashv f \cdot$	$(A \rightarrow C) \rightarrow (B \rightarrow C)$	$f f^\circ$	$1 \in f f^\circ$	$f^\circ f \in 1$	$f^\circ (XuY) = f^\circ X u f^\circ Y$	$f (X \eta Y) = f X \eta f Y$	$f f^\circ f = f$	$f^\circ f f^\circ = f^\circ$
$i \dashv E$	$\text{Map} \hookrightarrow \text{Rel}$	P	$\{\cdot\} : A \rightarrow P \ A$	$\exists : P \ B \rightarrow B$	—	—	$\Lambda(\exists) = 1$	$\Lambda(R) \exists = R$
$\cdot \exists \dashv \Lambda$	$\text{Map}(A, P \ B) \rightarrow (A \rightarrow B)$	1	$\Lambda(f \exists) = f$	$\Lambda(R) \exists = R$	—	—	$\Lambda(f \exists) \exists = f \exists$	$\Lambda(\Lambda(R) \exists) = \Lambda(R)$
$\Lambda \dashv \cdot \exists$	$(A \rightarrow B) \rightarrow \text{Map}(A, P \ B)$	1	$\Lambda(R) \exists = R$	$\Lambda(f \exists) = f$	—	—	$\Lambda(\Lambda(R) \exists) = \Lambda(R)$	$\Lambda(f \exists) \exists = f \exists$

§1.2. Composing adjunctions

(1.2a)

$$\begin{array}{c}
\boxed{\frac{X}{f(R/T)}} \xLeftrightarrow{f^\circ \cdot \dashv f \cdot} \boxed{\frac{f^\circ X}{R/T}} \xLeftrightarrow{\cdot T \dashv /T} \boxed{\frac{(f^\circ X) T}{R}} \\
\text{f a map} \\
\\
\xLeftrightarrow{\text{associativity}} \boxed{\frac{f^\circ (X T)}{R}} \xLeftrightarrow{f^\circ \cdot \dashv f \cdot} \boxed{\frac{X T}{f R}} \xLeftrightarrow{\cdot T \dashv /T} \boxed{\frac{X}{(f R)/T}} \\
\\
\xRightarrow{\text{indirect equality}} \boxed{\frac{f(R/T)}{(f R)/T}}
\end{array}$$

	$\cdot T$	$T \cdot$	$\cdot g$	$g^\circ \cdot$	Tn	$^\circ$	$\circ \triangleleft$	Δ	(1.2b)
$\cdot S$	$R/(ST) = (R/T)/S$	$T \setminus (R/S) = (T \setminus R)/S$	$R/(Sg) = (Rg^\circ)/S$	$g(R/S) = (gR)/S$	—	$(Y/S)^\circ = S^\circ \setminus (Y^\circ)$	$(RS)^\wedge = R^\wedge (S \otimes 1)$	$(T_1 n T_2)/S = T_1/S n T_2/S$	
$S \cdot$	$S \setminus (R/T) = (S \setminus R)/T$	$(TS) \setminus R = S \setminus (T \setminus R)$	$S \setminus (Rg^\circ) = (S \setminus R)g^\circ$	$(g^\circ S) \setminus R = S \setminus (gR)$	—	$(S \setminus Y)^\circ = Y^\circ/S^\circ$	$((S \otimes 1)R)^\wedge = S \ R^\wedge$	$S \setminus (T_1 n T_2) = S \setminus T_1 n S \setminus T_2$	
$\cdot f$	$R/(fT) = (R/T)f^\circ$	$T \setminus (Rf^\circ) = (T \setminus R)f^\circ$	$(fg)^\circ = g^\circ f^\circ$	—	—	$(fY)^\circ = Y^\circ f^\circ$	—	$(T_1 n T_2)f^\circ = T_1 f^\circ n T_2 f^\circ$	
$f^\circ \cdot$	$f(R/T) = (fR)/T$	$(Tf^\circ) \setminus R = f(T \setminus R)$	—	$(fg)^\circ = g^\circ f^\circ$	—	$(fY)^\circ = Y^\circ f^\circ$	—	$f(T_1 n T_2) = fT_1 n fT_2$	
Rn	—	—	—	—	$(RnT) \Rightarrow Y = R \Rightarrow (T \Rightarrow Y)$	$(R \Rightarrow Y)^\circ = R^\circ \Rightarrow (Y^\circ)$	—	$R \Rightarrow (T_1 n T_2) = (R \Rightarrow T_1) n (R \Rightarrow T_2)$	
$^\circ$	$(Y/T)^\circ = T^\circ \setminus (Y^\circ)$	$(T \setminus Y)^\circ = Y^\circ/T^\circ$	$(gY)^\circ = Y^\circ g^\circ$	$(gY)^\circ = Y^\circ g^\circ$	$(T \Rightarrow Y)^\circ = T^\circ \Rightarrow (Y^\circ)$	$Y^\circ = Y$	$(R^\wedge)^\circ = (R^\circ \otimes 1)(1 \otimes \triangleright \circ)$ both bends: the $^\circ$ section's display	$(T_1 n T_2)^\circ = T_1^\circ n T_2^\circ$	
$\circ \triangleleft$	$(RT)^\wedge = R^\wedge (T \otimes 1)$	$((T \otimes 1)R)^\wedge = T \ R^\wedge$	—	—	—	$(R^\wedge)^\circ = (R^\circ \otimes 1)(1 \otimes \triangleright \circ)$ both bends: the $^\circ$ section's display	—	—	
u	$(X_1 u X_2)T = X_1 T u X_2 T$	$T(X_1 u X_2) = TX_1 u TX_2$	$(X_1 u X_2)g = X_1 g u X_2 g$	$g^\circ(X_1 u X_2) = g^\circ X_1 u g^\circ X_2$	$Tn(X_1 u X_2) = (TnX_1)u(TnX_2)$	$(X_1 u X_2)^\circ = X_1^\circ u X_2^\circ$	—	—	
$\perp$	$\perp T = \perp$	$T \perp = \perp$	$\perp g = \perp$	$g^\circ \perp = \perp$	$Tn \perp = \perp$	$\perp^\circ = \perp$	—	—	

§1.3. Adjoint triples

Beyond $u \dashv \Delta \dashv n$, which §1.1 already carries as two rows, the table holds one adjoint triple with content: $\exists_f \dashv f^* \dashv \forall_f$ along a map f , once on each side of the composite. $f : A \rightarrow B$ throughout this subsection.

$$\begin{aligned} \cdot f &\dashv \cdot f^\circ \dashv /f^\circ \\ f^\circ \cdot &\dashv f \cdot \dashv f \setminus \end{aligned} \quad (1.3a)$$

The first two links of each chain are the map shunting rules, and the third is §1.1's division row read at a chosen divisor: $\cdot S \dashv /S$ at $S := f^\circ$, and $S \cdot \dashv S \setminus$ at $S := f$.

The third link is not the first over again. For a map f , $/f$ collapses to $\cdot f^\circ$; being a map is exactly what that collapse spends $R/(f \ S) = (R/S) \ f^\circ$. — f° is not a map, so $/f^\circ$ does not collapse in turn, and the three operators stay distinct. On $\text{Rel}(\mathbb{I}, A) = P \ A$ they are the image triple:

operator	acts by	name	(1.3b)
$\cdot f$	$S \mapsto f[S]$	direct image, $\exists_f$	
$\cdot f^\circ$	$T \mapsto f^{-1}[T]$	inverse image, f^*	
$/f^\circ$	$S \mapsto \{b : f^{-1}(b) \subseteq S\}$	$\forall_f$	

§1.1's $\mathcal{D} \dashv \cdot T$ is this same chain along the projection $A \otimes B \rightarrow A$, read through $\text{Rel}(A, B) = P(A \otimes B)$: $\mathcal{D}R = \{(a, a) : \exists b. aRb\}$, AT is the relation that ignores b altogether, and the third link is $R \mapsto 1 \cap R/T = \{(a, a) : \forall b. aRb\}$.

$$\mathcal{D} \dashv \cdot T \dashv 1n\cdot/T \quad (1.3c)$$

Three is where it stops, and one map breaks both ends. Take $A = \{a_1, a_2\}$ and $B = \{b\}$: $f[\{a_1\}] \cap f[\{a_2\}] = \{b\}$ while $f[\{a_1\} \cap \{a_2\}] = \emptyset$, so $\cdot f$ does not preserve meets and has no left adjoint, and the same two subsets give $\forall_f(\{a_1\} \cup \{a_2\}) = \{b\}$ against $\forall_f\{a_1\} \cup \forall_f\{a_2\} = \emptyset$, so $/f^\circ$ does not preserve joins and has no right adjoint. A monic f restores the binary case and no more — τf still reaches only $\text{im } f$, and $\forall_f \perp = B \setminus \text{im } f$ is still not $\perp$. The chain extends only when f° is a map as well, and then $/f^\circ = \cdot f^{\circ\circ} = \cdot f$ and it repeats forever. For an S with neither S nor S° a map there is no triple at all: $\cdot S \dashv /S$ is two links and stops.

The rows that chain forever say nothing by it: $^\circ$ is an order-isomorphism of hom-posets, so it is adjoint to itself on both sides and $^\circ \dashv \circ \dashv \circ \dashv \circ$ has no end, and §1.1's other row of that kind, $\circ \triangleleft \dashv \triangleright \circ$, goes the same way.

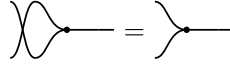
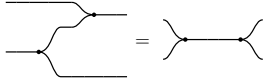
§2. Relations

$\mathbf{Rel}$ is a poset-enriched category with $(\otimes, \circ)$ where $\otimes$ is commutative cartesian product, and converse $\circ \dashv \circ$, and

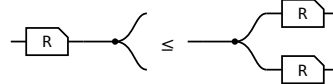
$\mathbf{C}: (\triangleright : A \otimes A \rightarrow A, \circ : \mathbb{I} \rightarrow A)$ a commutative monoid with $\mathbf{C}^\circ \dashv \mathbf{C}$. We write $\mathbf{C}^\circ$ as $(\triangleleft : A \rightarrow A \otimes A, \circ : A \rightarrow \mathbb{I})$



$\triangleright$ associative



$\triangleright$ commutative



$\circ$ is its unit

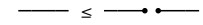
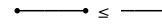
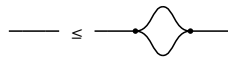


Frobenius, one half — the other is its $\circ$

$R\triangleleft \leq \triangleleft(R\otimes R)$

$R\circ \leq \circ$

§2.1. $\forall$ object A , $(A, \triangleleft, \circ) \dashv (A, \triangleright, \circ)$



$\triangleright \triangleleft \leq 1$

$1 \leq \triangleleft \triangleright$

$\circ \circ \leq 1$

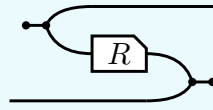
$1 \leq \circ \circ$

The last of these is the only one that makes a picture **bigger**, and it is worth a name: **a wire is below the cut wire**. In $\mathbf{Rel}$ it reads $\{(a, a)\} \subseteq A \times A$ — cut a wire and its two ends stop having to agree, so cutting can only add pairs. It is the one weakening this calculus gives away for free.

§3. $^\circ : \mathcal{C}^{\text{op}} \rightarrow \mathcal{C}$ is a 2 functor

(3a)

$^\circ$ is primitive, part of the data the first section lists. It turns both of R 's wires round, and the Frobenius structure DRAWS that — the picture and the formula below are R° , not its definition:



$$R^\circ = (\circlearrowleft \triangleleft \otimes 1) (1 \otimes R \otimes 1) (1 \otimes \triangleright \circlearrowright)$$

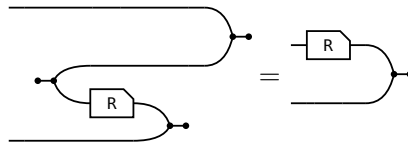
where $\circlearrowleft \triangleleft : \mathbb{I} \rightarrow a \otimes a$ opens a pair of wires out of nothing and $\triangleright \circlearrowright : b \otimes b \rightarrow \mathbb{I}$ closes one, so the input of R° is where the output of R was. Taking that formula as the definition would now be circular: $\triangleleft$ and $\circlearrowright$ are $\triangleright^\circ$ and $\circlearrowleft^\circ$. The laws of $^\circ$ are that it is a contravariant 2-functor $^\circ : \mathcal{C}^{\text{op}} \rightarrow \mathcal{C}$:

$$(i) \ 1^\circ = 1 \quad (ii) \ (R \ S)^\circ = S^\circ R^\circ \quad (iii) \ (R \otimes S)^\circ = R^\circ \otimes S^\circ \quad (iv) \ R \leq S \text{ implies } R^\circ \leq S^\circ$$

§3.1. The slide

The one rule (ii) and (iii) use, and each of them uses it twice. A converse facing a merge on the lower strand is the box itself, upright, on the upper one:

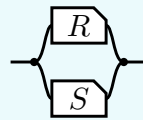
(3.1a)



R° is DEFINED as the bending of $(R \otimes 1) \triangleright \circlearrowright$, so the slide claims only that unbending it gives that back — and unbending undoes bending for every arrow. That is the snake above with a passenger: the $\circlearrowleft \triangleleft$ bends the a strand down and the $\triangleright \circlearrowright$ brings it back up, while the b strand rides through untouched. Nothing else is spent below.

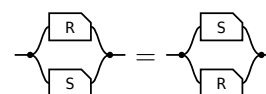
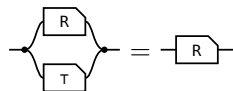
§4. $\cap$ is a commutative idempotent monoid on every hom-set

The **meet** of $R, S : a \rightarrow b$, the paper's **convolution**, is $R \cap S := \triangleleft (R \otimes S) \triangleright$ — copy the input, run R and S on the two copies, merge the results — so what comes out is what both of them do.

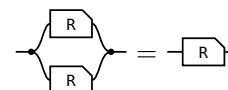
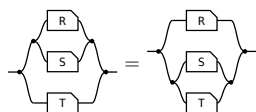


transcribed: a definition has no statement to export

On every hom-set it is associative, commutative and idempotent, with unit the maximal arrow $\tau = \multimap \multimap$ (the paper's Lemma 4.11).



unit: one half of τ per end — the merge's unit law absorbs the $\multimap$, the copy's counit law the $\multimap$ **commutative:** σ crosses $R \otimes S$ by naturality and is absorbed by cocommutativity and commutativity



associative: coassociativity and associativity; $\otimes$ re-brackets for nothing, **idempotent:** the one that is not bookkeeping — the lax copy law is the whole of it, worked in allegory2

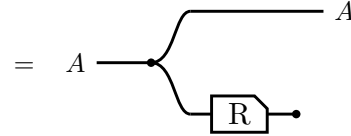
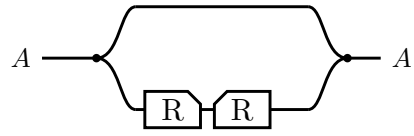
So $\leq$ is the order this monoid induces. $R \cap S \leq R$ comes from the unit, and idempotency turns anything under both S and T into something under $S \cap T$, since $R = R \cap R \leq S \cap T$.

And one law relating $\cap$ to composition, which is **not** an equation:

semi-distributivity, and what supplies it	picture
$R (S \cap T) \sqsubseteq R S \cap R T$ — the lax copy law. Equality exactly when R is single valued: the Maps section's $F (R \cap S) = F R \cap F S$.	

§5. Domain and range

The **domain** $\text{Dom } R \triangleq 1 \cap R R^\circ$ and the **range** $\text{Ran } R \triangleq \text{Dom } R^\circ$.



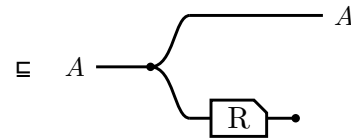
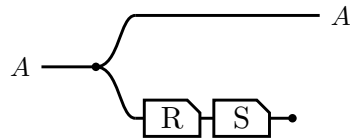
$\triangleright$ lands the return leg back on the value $\triangleleft$ handed out,
so $R^\circ \triangleright$ cuts to $\circ$ — Frobenius

Running R and throwing the result away leaves only the fact that R could fire, and $\text{Ran } R$ is the same picture with the box mirrored. In Rel both steps are $\{(a, a) : \exists b. a R b\}$.

$\text{Dom } R \triangleq 1 \cap R R^\circ$
$\text{Dom } R \subseteq A \iff R \subseteq A R, \text{ for } A \text{ coreflexive}$
$\text{Dom } (R S) \subseteq \text{Dom } R$
$\text{Dom } (R \cap S) = 1 \cap S R^\circ$
$R \text{ entire} \iff \text{Dom } R = 1 \iff 1 \subseteq R R^\circ$
$R \text{ simple} \iff R^\circ R \subseteq 1$
$R \text{ a map} \iff R \text{ entire and simple}$
$R, S \text{ entire} \implies R S \text{ entire} \text{ — likewise simple, likewise maps}$
$R S \text{ entire} \implies R \text{ entire}$

§5.1. Sliding the discard

$\text{Dom } (R S) \subseteq \text{Dom } R$, and a single glyph for Dom would have nothing to slide: with the box and the discard drawn apart, the law is one dot walking back along the lower strand.



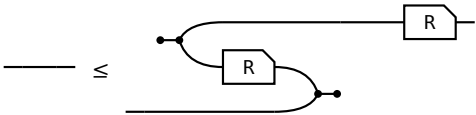
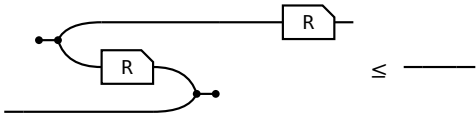
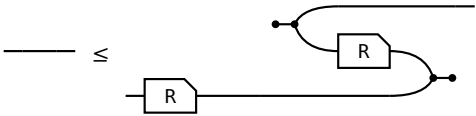
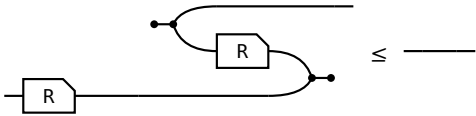
$S \circ \subseteq \circ$, the lax axiom for $\circ$ in the first section
— the discard slides back past S

Equality is $S \text{ entire}$, which is the same picture read as $\text{Dom } R = 1 \iff R \text{ entire}$.

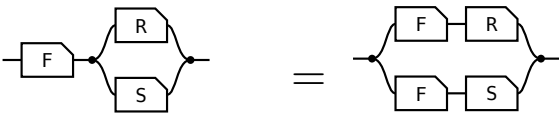
§6. Maps

Every arrow is lax for $\triangleleft$ and for $\multimap$ by axiom, and for $\triangleright$ and $\multimap$ by taking $\circ$ of those two. All four hold for **every** arrow, their REVERSES do not, and each reverse holding is a property of the arrow. The right-hand column is the **adjoint** form, which is the definition used here because it is literally the allegory's **Simple** and **Entire**; that the two forms agree is a separate theorem, not proved here.

(6a)

 surjective $1 \sqsubseteq R^\circ R$, that is, R° entire.	 single valued $R^\circ R \sqsubseteq 1$
 entire $1 \sqsubseteq R R^\circ$. With single valued, a map.	 injective $R R^\circ \sqsubseteq 1$

(6b)

law	picture
$F (R \cap S) = F R \cap F S$ F single valued F = people x may ask, R = admires a, S = admires b. Single valued means one person, so the sides agree.	 one person who admires both a at A, b at B

§7. / is all of

(7a)

$$x (R/S) y \iff \forall p. y S p \rightarrow x R p$$

$$x (S \backslash R) y \iff \forall p. p S x \rightarrow p R y$$

/ compares images: $S(y) \subseteq R(x)$. $\backslash$ compares preimages: $S^\circ(x) \subseteq R^\circ(y)$.

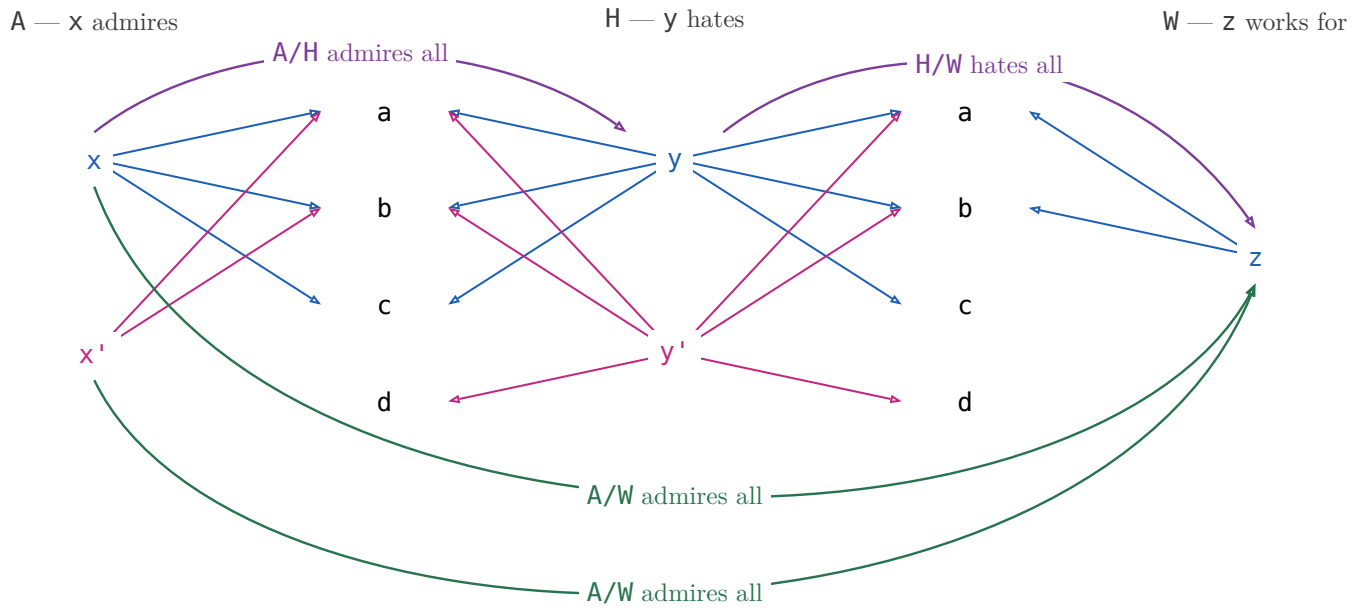
example: A admires, H hates, W works for

$x (A/H) y$ — x admires everyone y hates.

$x (H \backslash A) y$ — everyone who hates x admires y .

§7.1. $(R/S)(S/W) \sqsubseteq R/W$

(7.1a)



$$x (A/H) y \text{ — } x \text{ admires everyone } y \text{ hates}$$

$$y (H/W) z \text{ — } y \text{ hates everyone } z \text{ works for}$$

$$x (A/W) z \text{ — } x \text{ admires everyone } z \text{ works for}$$

(7.1b)

$(A/H)(H/W)$ is a path: $x \rightarrow y \rightarrow z$, and that is all of it. A/W also holds of x' , who admires everyone z works for — but x' does not admire everyone anybody hates, so nothing composes to it. The missing path is exactly the strictness of $(R/S)(S/W) \sqsubseteq R/W$.

$X \sqsubseteq R/S \iff X \ S \sqsubseteq R$ X is any X -to- Y pairing; one that only pairs X with a y such that x admires everyone y hates lies inside R/S , and R/S is the largest such.	
$X \sqsubseteq S \setminus R \iff S \ X \sqsubseteq R$ The mirror — divide on the left when X comes first.	
$(R/S) \ S \sqsubseteq R$ There is a y such that x admires everyone y hates, and p is one of the people y hates — then x admires p too. Strict at $S = \emptyset$: R/S is everyone, $(R/S) \ S = \emptyset$.	
$S \ (S \setminus R) \sqsubseteq R$ The mirror.	
associate: $R/(S_1 \ S_2) = (R/S_2)/S_1$ A friend's enemy is two hops: divide by the far end first.	
$(S_1 \ S_2) \setminus R = S_2 \setminus (S_1 \setminus R)$ The mirror.	
maps: $f \ (R/S) = (f \ R)/S$ Rename x before or after dividing — the licence to write $f \ R / S$.	
$R/(f \ S) = (R/S) \ f^\circ$ Rename y : a map leaves a denominator as f° outside the box.	
$(R/S) \ (S/W) \sqsubseteq R/W$ Someone who admires all of a hate-set that already covers everyone Z works for admires those people too.	
$1 \sqsubseteq R/R$ R/R runs admirer to admirer: each admires everyone they admire. Strict: two people who each admire only a and b admire each other's idols too, and still stay two people.	
$(R/R) \ (R/R) = R/R$ R/R is the preorder admires at least as much as , and a preorder is idempotent. Freyd writes $\sqsubseteq$; with $1 \sqsubseteq R/R$ above it is an equality.	
$(R/R) \ R = R$ Reaching p through someone whose idols x fully admires is reaching p directly, since x admires their own idols.	
$R/1 = R$ Dividing by 1 : p 's set is just $\{p\}$, so admiring all of it is admiring p .	
$R/(S_1 \cup S_2) = R/S_1 \ \cap \ R/S_2$ Admiring a combined hate-set is admiring each set in full.	
$S \setminus (R/W) = (S \setminus R)/W$ Which is why $S \setminus R/W$ needs no bracket.	

Fifteen laws, fifteen pictures, and not one shows a generator: η , υ , $\circ$ and composition are what the Frobenius generators build, and $/$ is none of those — it is posited, with nothing to unfold.

§8. $\frac{R}{S}$

$$\frac{R}{S} \triangleq (R/S) \cap (S/R)^\circ. \text{ In Rel } x \text{ and } y \text{ has the same image: } \forall p. (x R p \iff y S p)$$

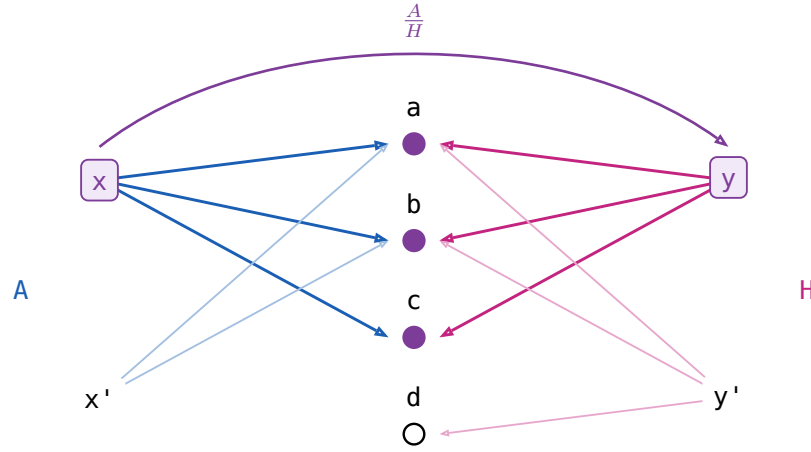
(8a)

$$\begin{aligned} x (A/H) y &\text{--- } x \text{ admires everyone } y \text{ hates} \\ x ((H/A)^\circ) y &\text{--- } x \text{ admires only people } y \text{ hates} \\ x ((A/H) \cap (H/A)^\circ) y &\text{--- } x \text{ admires only and all whom } y \text{ hates} \end{aligned}$$

(8b)

x admires exactly whom y hates: $/$ is **all of**, $\frac{R}{S}$ is **only all of**.

(8c)



law	the reading
$X \sqsubseteq \frac{R}{S} \iff XS \sqsubseteq R \text{ and } X^\circ R \sqsubseteq S$	X may pair x with y only when x admires exactly whom y hates. Both halves must typecheck, so the operation is partial .
$(\frac{R}{S})^\circ = \frac{S}{R}$	Matching is symmetric.
$\frac{R}{S} \frac{S}{W} \sqsubseteq \frac{R}{W}$	And transitive.
$\frac{R}{S} S \sqsubseteq R$	$(\exists y. x(\frac{R}{S})y \wedge ySp) \rightarrow xRp$ x only admires whom y hates $\frac{R}{S} S = \text{Dom}(\frac{R}{S})R$
$\frac{R}{R} R = R$	$(\exists y. x(\frac{R}{R})y \wedge yRp) \iff xRp$ x and y admire the same people $y = x$ always qualifies ($1 \sqsubseteq R\%R$ below)
$1 \sqsubseteq \frac{R}{R}$	$x(\frac{R}{R})y$ if x and y admires the same peoples.
$(\frac{R}{R})^2 = \frac{R}{R}$	So the relation admires the same people is an equivalence relation.
$X \sqsubseteq \frac{R}{R} \iff XR \sqsubseteq R$, for symmetric X	The largest symmetric arrow that leaves R alone.
$\frac{R}{1}$ is the simple part of R	The people who admire exactly one person and nobody else. It equals R only when R is simple, unlike $R/1 = R$.
$\text{Dom } \frac{R}{S} = 1 \cap (R/S)(S/R)$	Its domain is the domain of simplicity of R .

(8d)

§9. Freyd's Power allegories

(9a)

A **power allegory** is a division allegory with one unary operation on arrows, $\exists$ **epsilon**, subject to

$$\begin{aligned} \exists_R \square &= R\square, & \exists_R &= \exists_R \square \\ \mathbb{1} &\subseteq (R/\exists_R)(\exists_R/R) & \exists_R &\text{ is } \mathbf{thick} \\ (\exists_R/\exists_R) \cap (\exists_R/\exists_R)^\circ &\subseteq \mathbb{1} & \exists_R &\text{ is } \mathbf{straight} \end{aligned}$$

$R\square$ is R 's target, an identity arrow. For $R : A \rightarrow B$ write $\exists : P B \rightarrow B$, dropping the subscript.

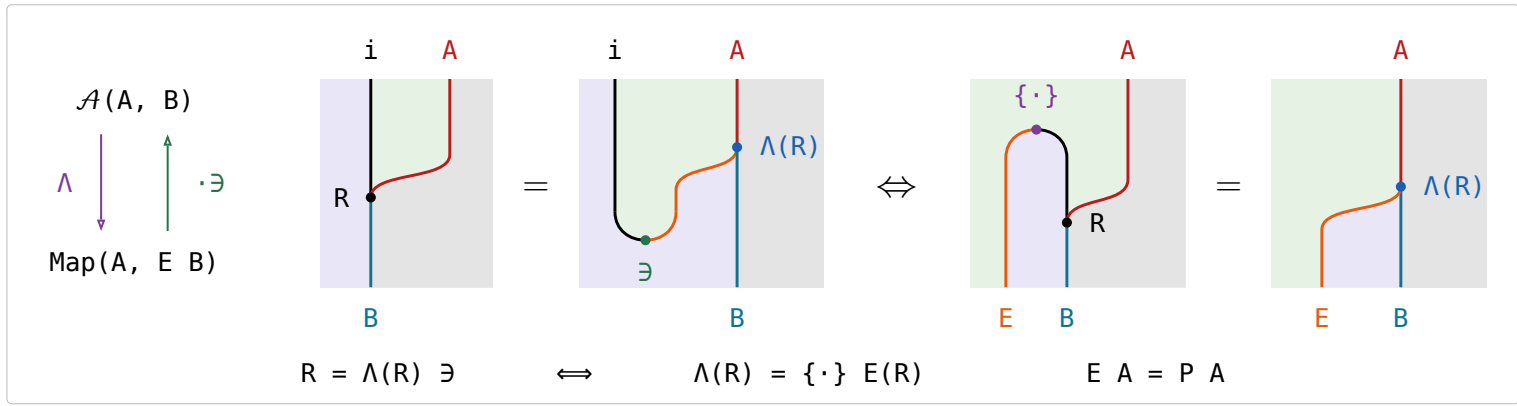
$$\epsilon \triangleq \exists^\circ : A \rightarrow P A$$

(9b)

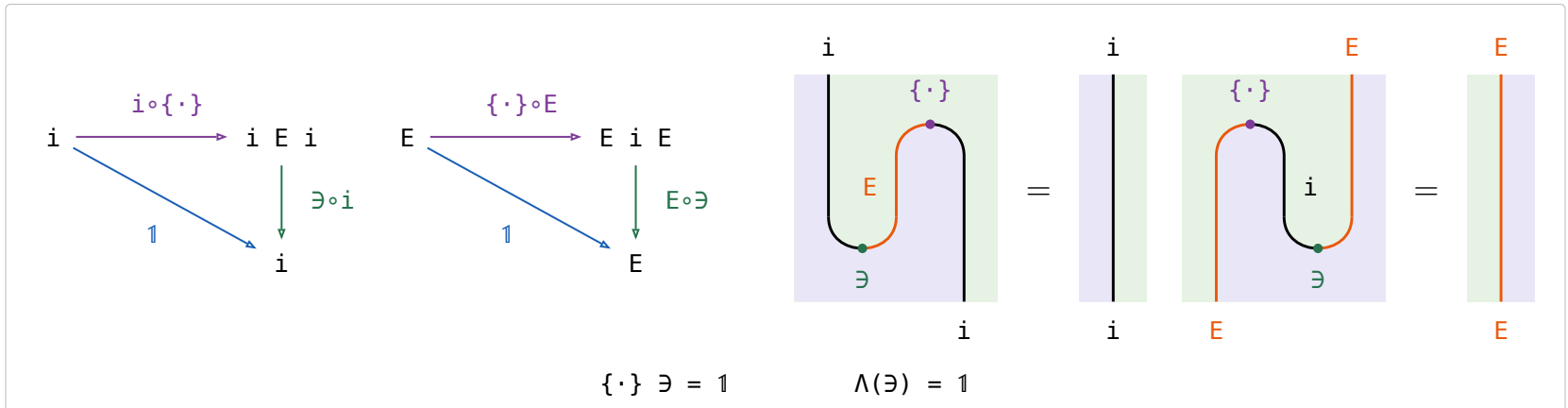
law	the reading
$\exists_R \square = R\square, \exists_R = \exists_R \square$	$\exists$ has the same target as R , and replacing R by the identity at that target leaves it unchanged: one $\exists$ per object, not per arrow.
$\exists$ is thick	Comprehension: every x has a set of exactly the people x admires. Equivalently every R factors as a map followed by $\exists$.
$\exists$ is straight , that is $\frac{\exists}{\exists} \subseteq \mathbb{1}$	Extensionality: two sets with the same people are the same set.
$\Lambda(R) \triangleq \frac{R}{\exists} : A \rightarrow P B$, for $R : A \rightarrow B$	convert a relation to a function. a $\Lambda(R) = \{b \mid a R b\}$, image of a
$\Lambda(R)$ is simple	$\exists$ is straight, and dividing by a straight arrow is simple. At most one set per x .
$\Lambda(R)$ is entire $\iff \exists$ thick	$\text{Dom } \frac{R}{\exists} = \mathbb{1} \cap (R/\exists)(\exists/R)$, the domain row above. At least one set per x , so $\Lambda(R)$ is a map .
$\Lambda(R) \exists = R$	Drawn in §10, where it is the counit's cancellation.
$\Lambda(R)$ is the only map with $\Lambda(R) \exists = R$	Two maps naming the same people name the same set — extensionality again.
$F \subseteq \Lambda(F \exists)$, F simple	A partial choice of sets is inside the total one.
$P A \triangleq$ source of $\exists$, the power-object	Notation, not a law: $\exists$ goes $P A \rightarrow A$, and $P A$ is simply a name for where it starts. Every subset of A .
$\{\cdot\} \triangleq \Lambda(1)$, the singleton map , monic	The one-person set. $\Lambda(1)\Lambda^\circ(1) \subseteq \frac{\mathbb{1}}{\exists} \frac{\exists}{\mathbb{1}} \subseteq \frac{\mathbb{1}}{\mathbb{1}} \subseteq \mathbb{1}$.
$\Lambda(\exists) = 1$	Make the set of a set, then read it back one level down. Straightness itself, and one of the two snake laws of §10; the other is $\{\cdot\} \exists = 1$.
fusion: $\Lambda(f R) = f \Lambda(R)$, f a map	Naturality of the unit, $f \{\cdot\} = \{\cdot\} (E(f))$, and that is the whole content of fusion. Drawn in §10.
$\Lambda(f) = f \{\cdot\}$, f a map	Rename first or take singletons first — the fusion row above at $R = 1$.
$\frac{R}{S} = \Lambda(R)\Lambda^\circ(S)$	x and y match exactly when $\Lambda(R)x$ and $\Lambda(S)y$ are one and the same set.
$E(R) \triangleq \Lambda(\exists R)$	$E(R) : P A \rightarrow P B$, image of a set of A
$\Lambda(R) = \{\cdot\} E(R)$	$\{\cdot\} : x \mapsto \{x\}$
$\Lambda(S) E(R) = \Lambda(S R)$	absorption
subset $\triangleq \exists/\exists : P A \rightarrow P A$	x subset $y \iff \forall a. y \ni a \rightarrow x \ni a$, that is $y \subseteq x$: diagram order reverses B&dM's argument order along with the arrow. It models inclusion between sets, where $\subseteq$ is the allegory's own primitive comparing arrows (B&dM p. 106).

§10. $i \dashv E$ Power Allegory defined as adjunction

(10a)

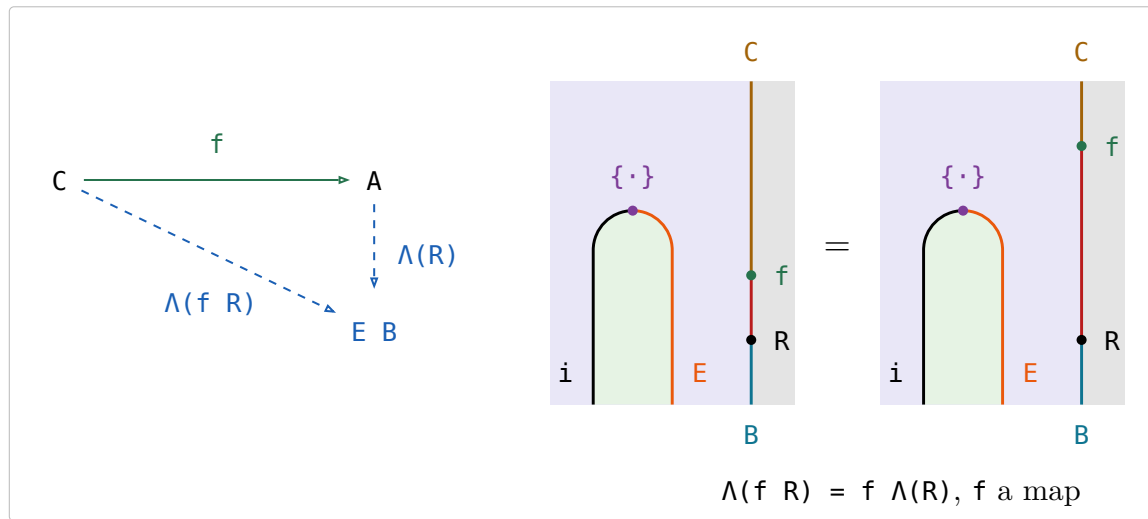


(10b)

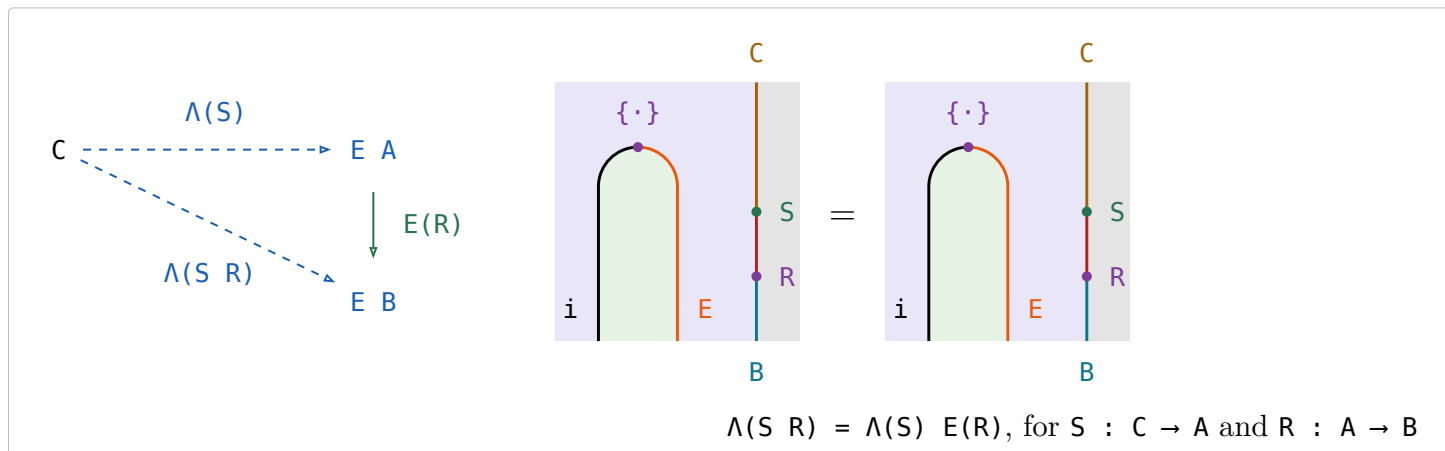


§10.1. Fusion law

(10.1a)



(10.1b)



scan line	the object on it	the bead just passed
the top edge	C	—
between $\{ \cdot \}$ and S	$i \circ E C$	$\{ \cdot \} : C \rightarrow E C$
between S and R	$i \circ E A$	S , inside the pair, so $E(S) : E C \rightarrow E A$
the bottom edge	$i \circ E B$	R , likewise $E(R) : E A \rightarrow E B$

(10.1c)

One string of beads, two groupings:

- The first two, then the third: $\{\cdot\} E(S) = \Lambda(S) : C \rightarrow E A$ followed by $E(R) : E A \rightarrow E B$.
- All three at once: $\{\cdot\} E(S) E(R) = \{\cdot\} E(S R) = \Lambda(S R) : C \rightarrow E B$.

Absorption holds for every S , so fusion is its case $S := f$, plus one step: absorption there reads $\Lambda(f R) = \Lambda(f) E(R)$, and turning $\Lambda(f) E(R)$ into $f \Lambda(R)$ needs $\Lambda(f) = f \{\cdot\}$, which is (10.1a) at $R = 1$. That step is the unit's naturality, and it is all fusion adds.

§10.2. $\Lambda(R) = (R/\exists) \cap (\exists/R)^\circ$

$$\begin{array}{c}
 \boxed{\frac{g^\circ f}{\Lambda(R)}} \xleftrightarrow{\bar{f}^\circ \cdot \dashv f \cdot} \boxed{\frac{\underline{f}}{g \Lambda(R)}} \xleftrightarrow{\text{fusion}} \boxed{\frac{\underline{f}}{\Lambda(g R)}} \xleftrightarrow{\cdot \exists \dashv \Lambda} \boxed{\frac{\underline{f \exists}}{g R}} \\
 \text{f, g maps} \\
 \begin{array}{ccccc}
 & \boxed{\frac{f \exists}{g R}} & \xleftrightarrow{\bar{f}^\circ \cdot \dashv f \cdot} & \boxed{\frac{g^\circ f \exists}{R}} & \xleftrightarrow{\cdot S \dashv / S} & \boxed{\frac{g^\circ f}{R/\exists}} \\
 \xleftrightarrow{\text{antisymmetry}} & \boxed{\frac{g R}{f \exists}} & & \boxed{\frac{f^\circ g R}{\exists}} & & \boxed{\frac{f^\circ g}{\exists/R}}
 \end{array} \\
 \xleftrightarrow{\circ, \text{meet}} \boxed{\frac{g^\circ f}{(R/\exists) \cap (\exists/R)^\circ}}
 \end{array} \tag{10.2a}$$

§11. Relator

Every hom-set of an allegory is a poset, so an allegory is a **locally posetal 2-category**: the 2-cell from R to S is $R \sqsubseteq S$. A **relator** $F : \mathcal{C} \rightarrow \mathcal{D}$ is a 2-functor between allegories:

$$F(1) = 1 \quad F(R \circ S) = F(R) \circ F(S) \quad R \sqsubseteq S \implies F(R) \sqsubseteq F(S)$$

Preserving $\circ$ is **not** asked for — $\circ$ is an identity-on-objects involution $\mathcal{C}^{\text{op}} \rightarrow \mathcal{C}$, no part of the 2-category.

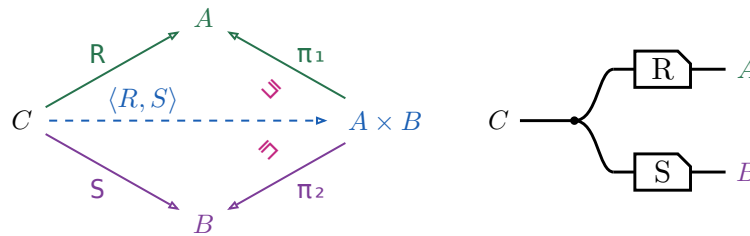
the statement	B&dM	(11b)
For f a map, $F(f)$ is a map and $F(f^\circ) = F(f)^\circ$.	Lemma 5.1	
Over a tabular allegory a functor is a relator $\iff$ it preserves $\circ$.	Theorem 5.1	
$F(R^\circ) = F(R)^\circ$ for every R , so $F(R)^\circ$ needs no bracket.	after Theorem 5.1, p. 113	
Two relators agreeing on maps are equal.	Corollary 5.1	
$F(X \cap Y) = F(X) \cap F(Y)$ for X, Y coreflexive.	Ex 5.2	
$F(R \cap S) \sqsubseteq F(R) \cap F(S)$, and strictly.	Ex 5.2, the restriction	
$F(\text{Dom } R) = \text{Dom}(F(R))$ for F preserving $\circ$.	—	

The **power relator** $P \multimap x \mapsto P(R) \ y \iff (\forall a \in x. \exists b \in y. a R b) \wedge (\forall b \in y. \exists a \in x. a R b)$ — is where the fourth is strict: for $R = \{(a_1, b_1), (a_2, b_2)\}$ and $S = \{(a_1, b_2), (a_2, b_1)\}$ the pair $(\{a_1, a_2\}, \{b_1, b_2\})$ is in $P(R) \cap P(S)$, while $R \cap S = \emptyset$.

§12. Fork $\langle R, S \rangle$

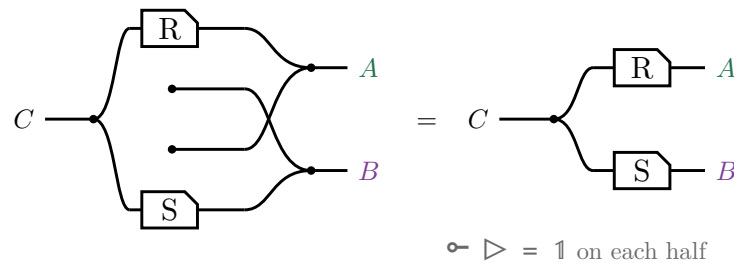
The **fork** of $R : \mathcal{C} \rightarrow A$ and $S : \mathcal{C} \rightarrow B$ is $\langle R, S \rangle \triangleq R \pi_1^\circ \cap S \pi_2^\circ$, where (π_1, π_2) is the tabulation of τ .

$$\langle R, S \rangle \pi_1 = (\text{Dom } S) R \quad \langle R, S \rangle \pi_2 = (\text{Dom } R) S$$



A domain is coreflexive, so $\langle R, S \rangle \pi_1 \sqsubseteq R$, with equality exactly when S is entire; for maps both triangles commute and $\langle f, g \rangle$ is unique. In $\mathbf{Rel}$, $c \in \langle R, S \rangle (a, b)$ iff $c R a$ and $c S b$ — copy c , then R on one strand and S on the other, which is $\triangleleft (R \otimes S)$ on the right.

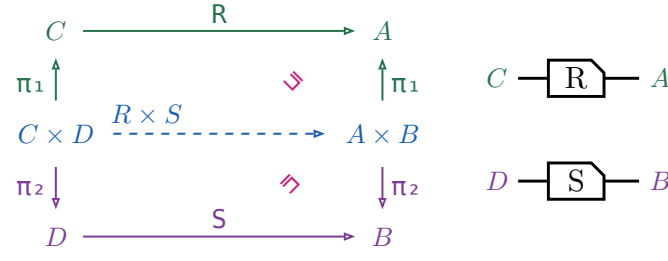
No $\circ$ survives the translation. $\pi_1 = 1 \otimes \multimap$ discards the second component, so $\pi_1^\circ = 1 \otimes \multimap$ **creates** one out of nothing, and $\cap$ is copy, run both, merge. Draw that and the created strands — the two dots with no left end, and the crossing they force — are merged against real ones, which is the monoid's unit law:



§12.1. Relational product $R \times S$

$R \times S \triangleq \langle \pi_1 R, \pi_2 S \rangle$, a relator in each argument but no longer a categorical product.

(12.1b)

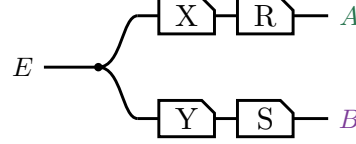


Right-then-up is $(R \times S) \pi_1$, up-then-right is $\pi_1 R$, and $(R \times S) \pi_1 \sqsubseteq \pi_1 R$, equality when S is entire. In $\mathbf{Rel}$, $(c, d) (R \times S) (a, b)$ iff $c R a$ and $d S b$ — two strands side by side, no copy dot: $R \times S = R \otimes S$.

§12.2. Absorption

For $X : E \rightarrow C$ and $Y : E \rightarrow D$, $\langle X, Y \rangle (R \times S) = \langle X R, Y S \rangle$. Both sides are this picture:

(12.2a)



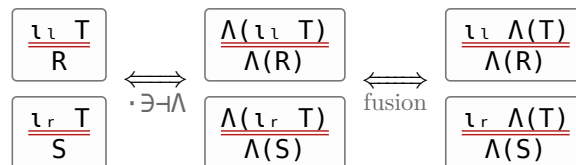
§13. Coproduct $[R, S]$

the statement	picture	(13a)
$[R, S] \triangleq \iota_l^\circ R \cup \iota_r^\circ S$ The tape is the union — a particle entering at $A + B$ takes exactly one branch — and the two mirrored boxes are what makes the branches disjoint.		
$[R, S] = [\Lambda(R), \Lambda(S)] \ni$		
$R + S \triangleq [R \iota_l, S \iota_r]$		
$\iota_l [R, S] = R$, $\iota_r [R, S] = S$, and $[R, S]$ is the only such arrow		
$\iota_l \iota_l^\circ = \mathbb{1} = \iota_r \iota_r^\circ$		
$\iota_l \iota_r^\circ = \perp = \iota_r \iota_l^\circ$		
$\iota_l^\circ \iota_l \cup \iota_r^\circ \iota_r = \mathbb{1}$		
$[U, V]^\circ [R, S] = U^\circ R \cup V^\circ S$		

§13.1. $[R, S] \triangleq [\Lambda(R), \Lambda(S)] \ni$

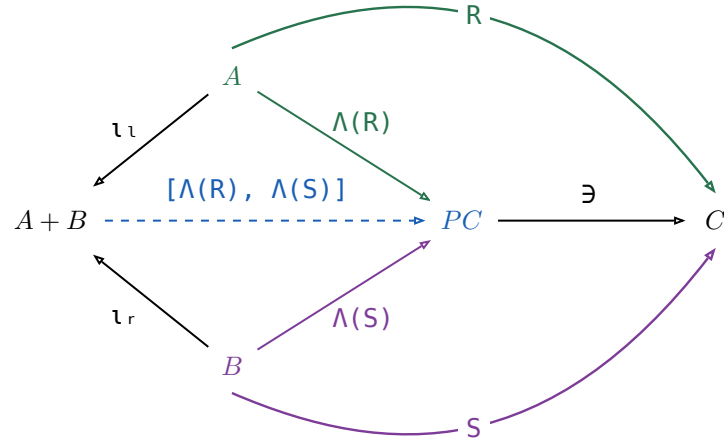
The universal-property row is not free: ι_l, ι_r were only ever asked to be a coproduct of **maps**. They stay one once every arrow is allowed because Λ sends an arrow $A \rightarrow C$ to a map $A \rightarrow P C$ reversibly, so the map coproduct can be applied underneath it. For any $T : A + B \rightarrow C$,

(13.1a)



$$\begin{array}{ccc} \xleftrightarrow{\text{coproduct of maps}} & \boxed{\frac{\Lambda(T)}{[\Lambda(R), \Lambda(S)]}} & \xleftrightarrow{\cdot \exists \dashv \Lambda} \boxed{\frac{T}{[\Lambda(R), \Lambda(S)] \exists}} \end{array}$$

(13.1b)



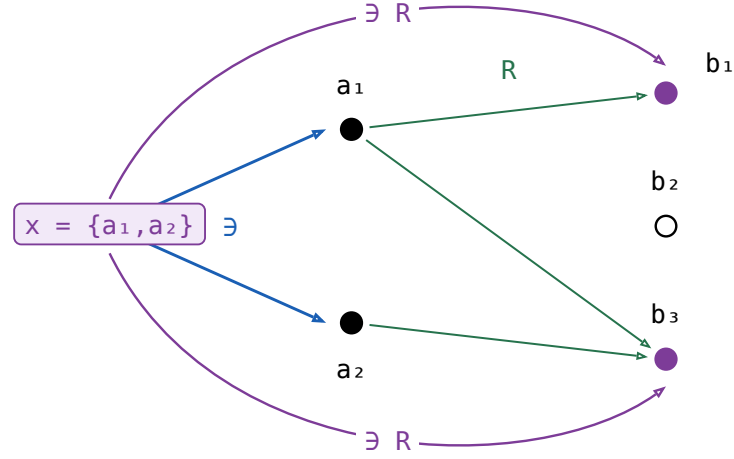
Nothing here holds only up to Ξ : every triangle commutes on the nose, which is the difference from the fork above. The border spells $[R, S] = [\Lambda(R), \Lambda(S)] \exists$, and pushing $\exists$ into the union that $[\cdot, \cdot]$ on maps already is turns that back into the definition, $(\iota_l \circ \Lambda(R) \cup \iota_r \circ \Lambda(S)) \exists = \iota_l \circ R \cup \iota_r \circ S$.

§14. The power relator $P(R)$

For $R : A \rightarrow B$, $P(R) \triangleq ((\exists R)/\exists) \cap ((\exists R^\circ)/\exists)^\circ : P A \rightarrow P B$.

$$x P(R) y \iff (\forall a \in x. \exists b \in y. a R b) \wedge (\forall b \in y. \exists a \in x. a R b)$$

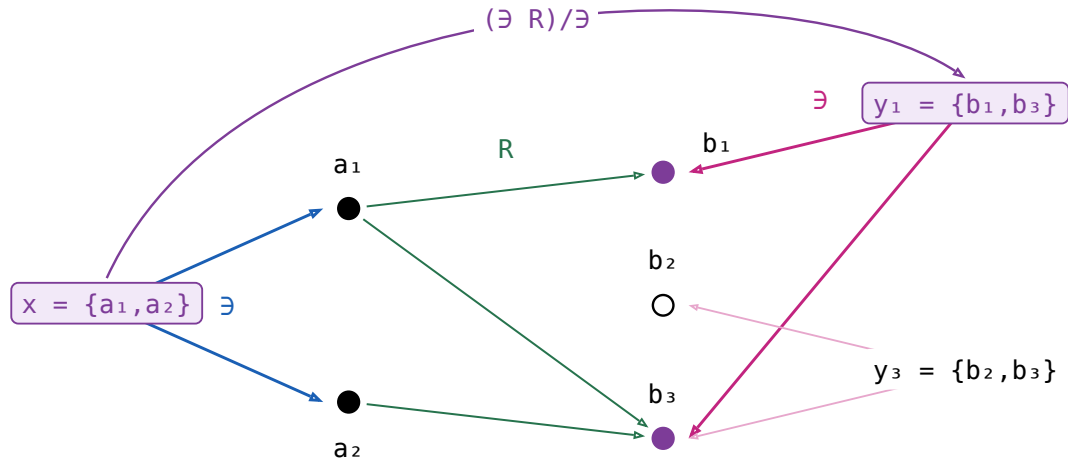
Every element of x is related by R to some element of y , and conversely.



$$\exists R : P A \rightarrow B \quad x (\exists R) b \iff \exists a \in x. a R b$$

one arc per element of B that x reaches, and b_2 gets none

Dividing by $\exists$ turns that into a relation between **sets**.

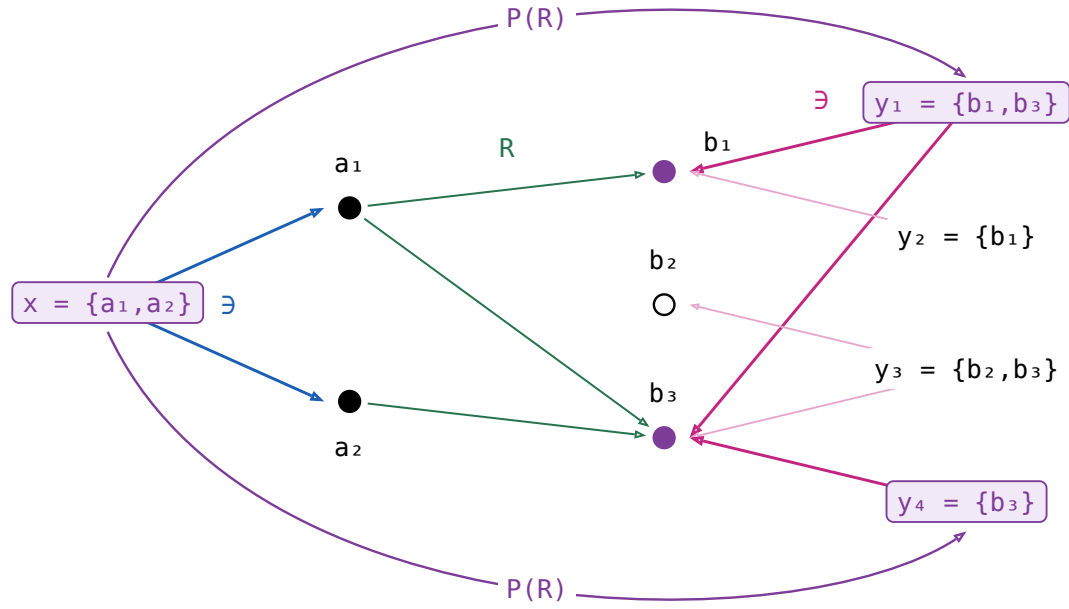


$$(\exists R)/\exists : P A \rightarrow P B \quad x ((\exists R)/\exists) y \iff \forall b \in y. \exists a \in x. a R b$$

y_3 is rejected: it names b_2 , which x does not reach

The other half of the meet is that clause with the sets swapped and R turned round: every element of x must reach y :

(14d)



$$((\exists R^\circ)/\exists)^\circ : P A \rightarrow P B \quad x ((\exists R^\circ)/\exists)^\circ y \iff \forall a \in x. \exists b \in y. a R b$$

y_2 is rejected too: a_2 's only image is b_3 , which y_2 does not name

(14e)

law	the reading
$X \sqsubseteq P(R) \iff X \ni \sqsubseteq \exists R$ and $X^\circ \ni \sqsubseteq \exists R^\circ$	One containment, and the same one at R° — which is the definition read off the two divisions. Hence $P(R^\circ) = P(R)^\circ$, and $R \sqsubseteq S \implies P(R) \sqsubseteq P(S)$.
$P(1) = \frac{\exists}{\exists} = 1$	The straightness axiom verbatim: extensionality is P 's unit law.
$P(f) = \Lambda(\exists f) = \frac{\exists f}{\exists}$, for f a map	In $\mathbf{Rel}$, $x P(f) y \iff y = \{f a \mid a \in x\}$. The half at f° says every $a \in x$ has its $f a$ on y ; f has just the one image per a , so that already says y contains everything x reaches, which is the fraction's second half. For a map the two definitions coincide.
$P(R S) = P(R) P(S)$	$\sqsupseteq$ is the division cancellation laws. $\sqsubseteq$ is the one law in this section that is not a calculation: it needs a tabulation of $P(R S)$.

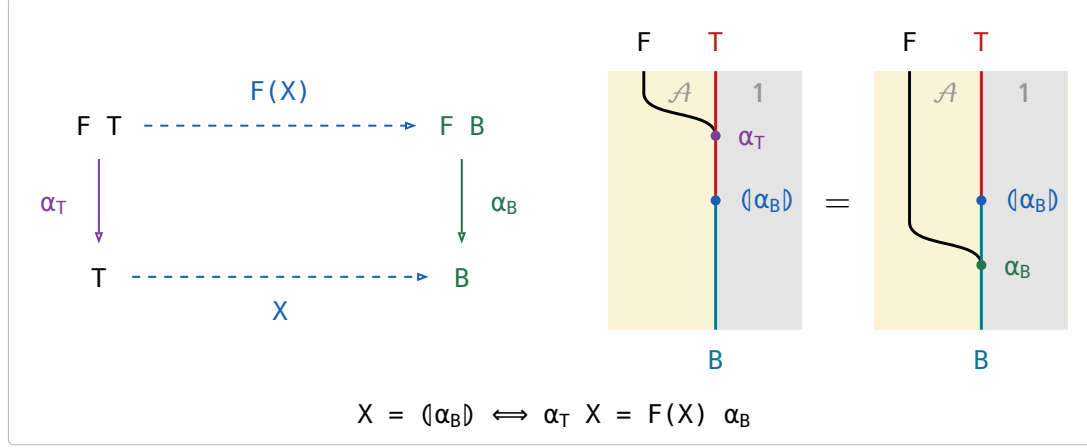
§15. Catamorphism

F a relator with an **initial algebra** $\alpha_T : F\ T \rightarrow T$ among the maps. For a relational algebra $\alpha_B : F\ B \rightarrow B$, the **catamorphism** $\langle \alpha_B \rangle : T \rightarrow B$ is the unique arrow with $\alpha_T \langle \alpha_B \rangle = F(\langle \alpha_B \rangle) \alpha_B$. Explicitly, for a relation $R : F\ B \rightarrow B$, $\langle R \rangle \triangleq \langle \wedge(F(\exists)\ R) \rangle \exists$, the inner $\langle \cdot \rangle$ being the catamorphism of a **map** algebra.

(15a)

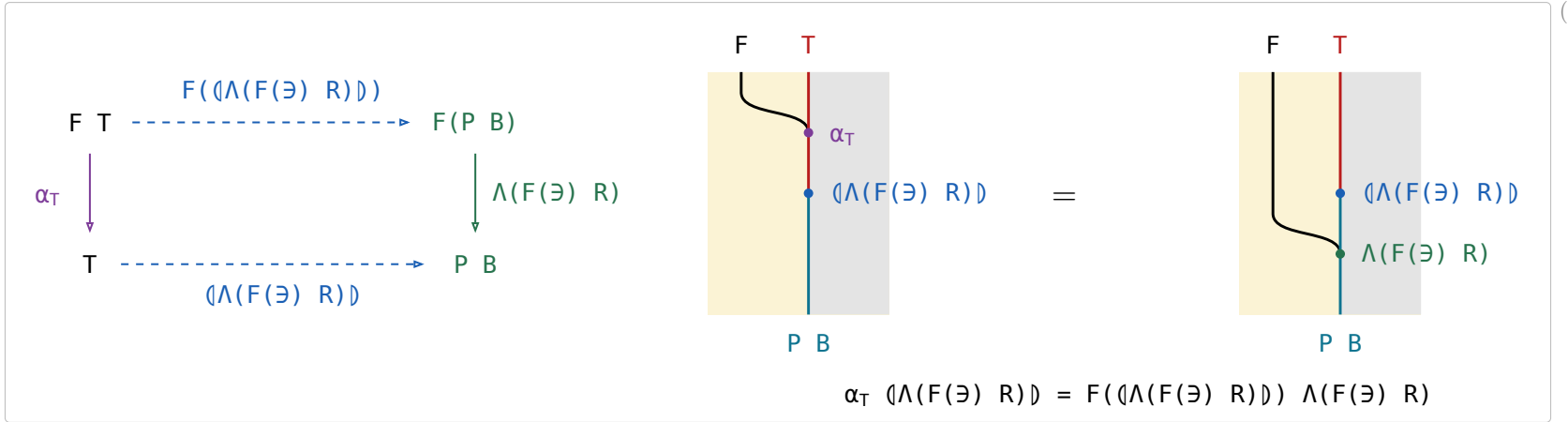
An F -algebra is a **weakened** natural transformation. A transformation $F \Rightarrow 1$ on $\mathcal{A}$ would need a component $F\ X \rightarrow X$ at every object and a commuting square at every arrow, but F -Algebra only need it works on T and B .

§15.1. The defining equation



(15.1a)

§15.2. $\langle R \rangle = \langle \wedge(F(\exists)\ R) \rangle \exists$



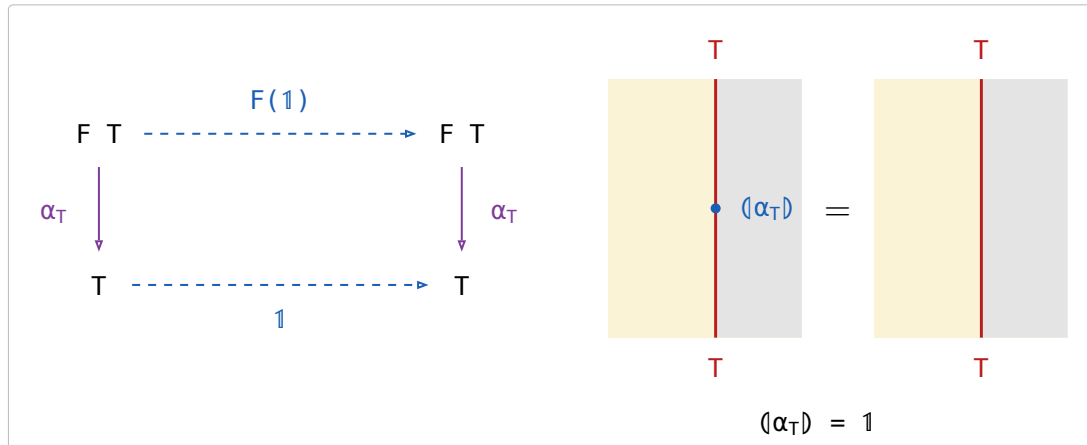
(15.2a)

$$\begin{array}{c}
 \boxed{\frac{\alpha_T X}{F(X) R}} \iff \boxed{\frac{\wedge(\alpha_T X)}{\wedge(F(X) R)}} \iff \boxed{\frac{\wedge(\alpha_T X)}{\wedge(F(\wedge(X) \exists) R)}} \\
 \iff \boxed{\frac{\alpha_T \wedge(X)}{F(\wedge(X)) \wedge(F(\exists) R)}} \iff \boxed{\frac{\wedge(X)}{\langle \wedge(F(\exists) R) \rangle}} \iff \boxed{\frac{X}{\langle \wedge(F(\exists) R) \rangle \exists}}
 \end{array}$$

relator, fusion twice catamorphism of maps $\cdot \exists \dashv \wedge$

(15.2b)

§15.3. Reflection

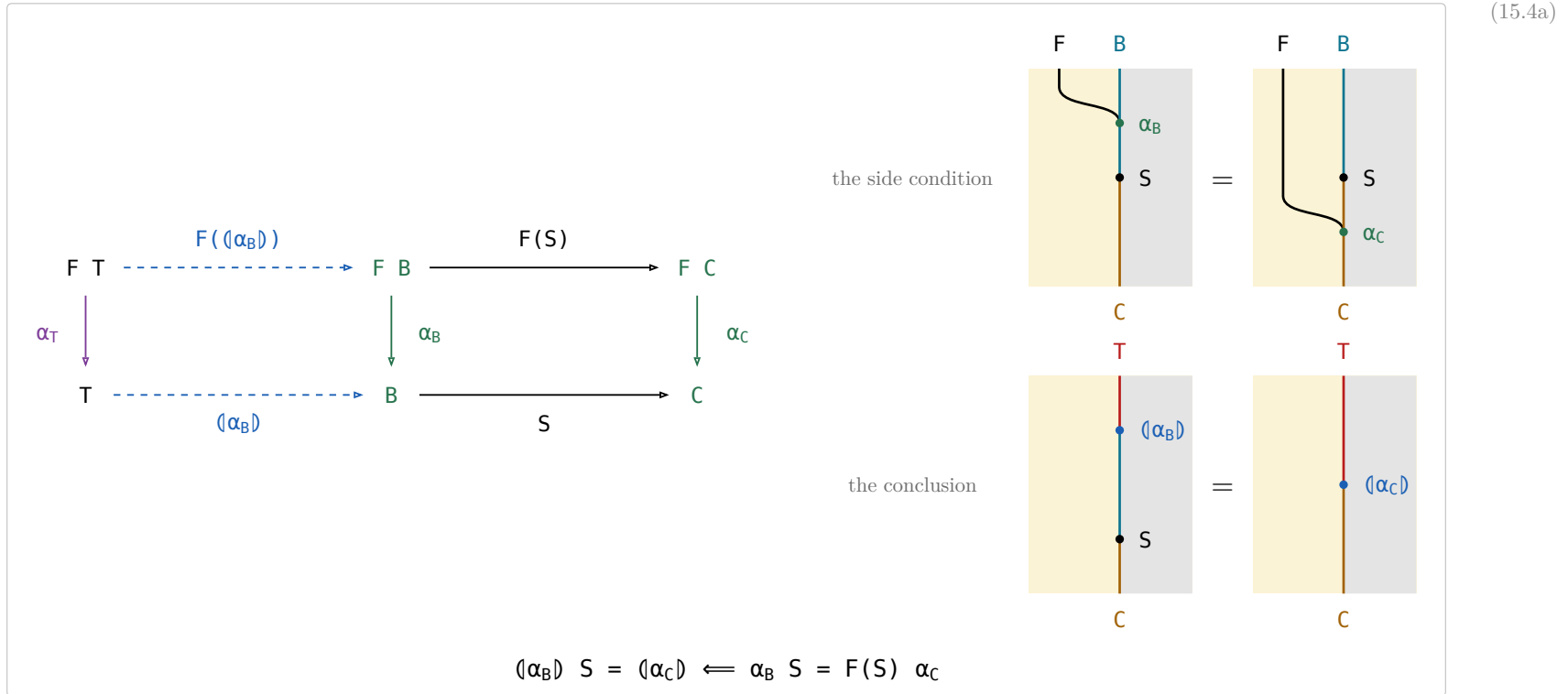


(15.3a)

Taking a value apart with α_T and putting it straight back is doing nothing.

§15.4. Fusion

Fusion rewrites $(\alpha_B) S$ through a second algebra $\alpha_C : F C \rightarrow C$ along an arrow $S : B \rightarrow C$.



The left square is (α_B) 's own defining square and the right one is the side condition, so the outer rectangle says $(\alpha_B) S$ satisfies the defining equation of (α_C) — and uniqueness finishes it. A fold followed by S has collapsed into a single fold, which is how an intermediate structure is got rid of.

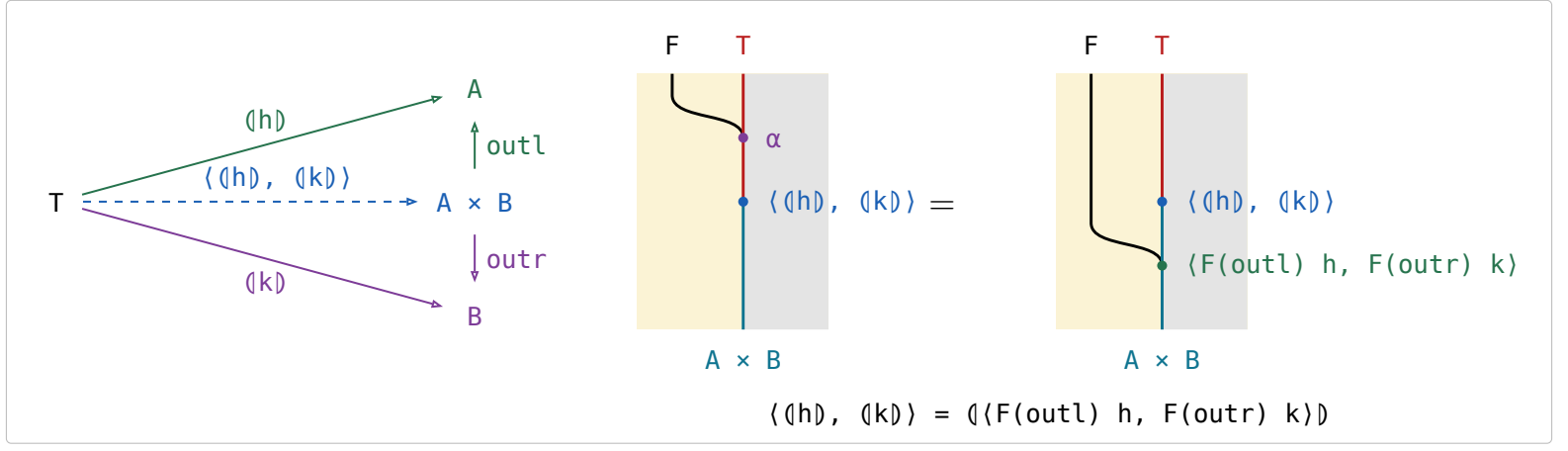
The side condition is (15.1a)'s picture again under $\alpha_T \mapsto \alpha_B$, $(\alpha_B) \mapsto S$, $\alpha_B \mapsto \alpha_C$: both say that the bead between the two merges is a homomorphism of F -algebras.

§15.5. Catamorphism in $\mathcal{S}et$

(15.5a)

name	definition	note
<code>listr</code>	<code>listr A ::= nil cons (A, listr A)</code>	The cons-lists over A, the datatype every row below folds (B&dM p. 55).
<code>sum</code>	<code>([zero, plus])</code>	<code>plus(a, b) = a + b</code> .
<code>length</code>	<code>([zero, outr succ])</code>	<code>outr</code> drops the head and keeps the count of the tail, <code>succ</code> adds one for the head.
<code>average</code>	<code>(sum, length) div</code>	<code>div(m, n) = m/n</code> , with <code>div(0, 0) = 0</code> so <code>average</code> is total. Traverses the list twice.
banana-split law	$\langle (h), (k) \rangle = \langle F(\text{outl}) h, F(\text{outr}) k \rangle$	Any fork of folds is a single fold, hence one traversal — F the base functor.
what it reduces to	$\alpha \langle (h), (k) \rangle = F(\langle (h), (k) \rangle) \langle F(\text{outl}) h, F(\text{outr}) k \rangle$	All that (15.1a) leaves to check: the fork satisfies the defining equation, and uniqueness finishes it.
the instance	<code>(sum, length) = ([zeros, pluss])</code>	<code>zeros = (zero, zero)</code> and <code>pluss(a, (b, n)) = (a + b, n + 1)</code> , so <code>average</code> runs in one pass.
<code>preds</code>	<code>preds : Nat -> [Nat], preds n = [n, n-1, ..., 1]</code>	B&dM Ex 3.6 (p. 57), posed and not answered there: apply Ex 3.4 to write <code>preds</code> as <code>(k) outl</code> .

(15.5b)



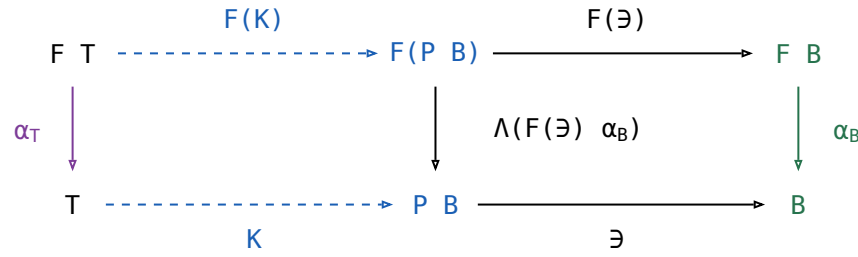
The other laws.

The two $\sqsubseteq$ **fusion** rows rewrite $\langle \alpha_B \rangle\ S$ through the same α_C and S . They are the only rows here that need the allegory **locally complete** — every hom-set a complete lattice — because they come from a least-fixed-point argument; the rest, the equality fusion (15.4a) included, needs only the initial algebra and the defining equation.

name	law	what it says
Lambek	$\alpha_T^\circ = \alpha_T^{-1}$, the initial algebra is an isomorphism	A constructor can be undone — α_T° takes a value apart into the parts it was built from.
Eilenberg–Wright	$\Lambda(\langle \alpha_B \rangle) = \langle \Lambda(F(\exists)\ \alpha_B) \rangle$	A relational fold is a deterministic fold of SETS: Λ pushes the nondeterminism into the power-object, where the fold is a map again.
Eilenberg–Wright	$\langle \alpha_B \rangle = \langle \Lambda(F(\exists)\ \alpha_B) \rangle\ \exists$	The same fact read back — fold deterministically into a set, then take a member of it.
fusion	$\langle \alpha_C \rangle \sqsubseteq \langle \alpha_B \rangle\ S \iff F(S)\ \alpha_C \sqsubseteq \alpha_B\ S$	Half of fusion: an inclusion between the two algebras is inherited by the folds.
fusion	$\langle \alpha_B \rangle\ S \sqsubseteq \langle \alpha_C \rangle \iff \alpha_B\ S \sqsubseteq F(S)\ \alpha_C$	The other half, with both inclusions turned around.

(15.5d)

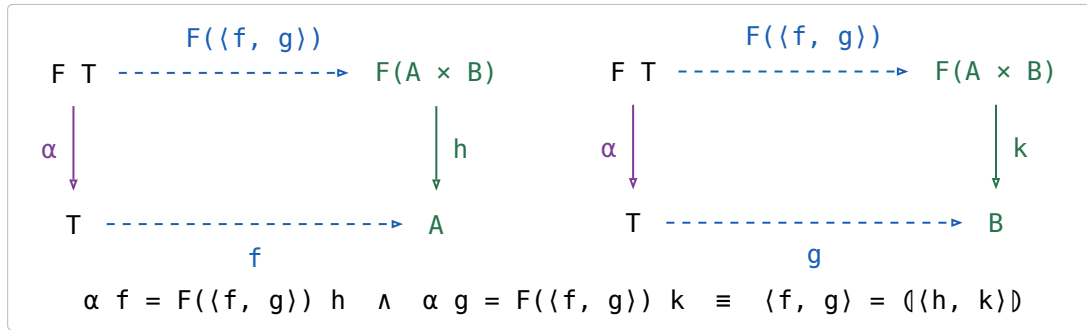
The two Λ rows, drawn:



The left square is that catamorphism's own defining square, $K = \langle \Lambda(F(\exists)\ \alpha_B) \rangle$, and the right one is Λ 's cancellation, so the outer rectangle says $K\ \exists$ satisfies the defining equation of $\langle \alpha_B \rangle$ — and uniqueness finishes it.

§15.5.1. Fokkinga's mutual recursion theorem

- $F\text{-Alg}(\mathcal{A})$ — the algebras and their homomorphisms — has binary products whenever $\mathcal{A}$ does, and the forgetful $U : F\text{-Alg}(\mathcal{A}) \rightarrow \mathcal{A}$ creates them: the product of $h : F A \rightarrow A$ and $k : F B \rightarrow B$ is carried by $A \times B$, with structure $\langle F(\text{outl}) h, F(\text{outr}) k \rangle : F(A \times B) \rightarrow A \times B$.
- outl and outr are homomorphisms out of it, and $\langle p, q \rangle : X \rightarrow A \times B$ is a homomorphism **iff** p and q both are — substitute the structure and compare the two forks component by component.
- The banana-split law of (15.5a) IS that product's universal property read at the initial algebra: $\langle h \rangle$ and $\langle k \rangle$ are the unique homomorphisms to (A, h) and (B, k) , so their fork is the unique homomorphism into the product, which is $\langle F(\text{outl}) h, F(\text{outr}) k \rangle$.
- Fokkinga's theorem is **strictly more general** and is not that product: in (15.5.1a) the two arrows leave $F(A \times B)$, not $F A$ and $F B$, so each sees BOTH components. The product is the case where they factor as $F(\text{outl}) h$ and $F(\text{outr}) k$; B&dM's Ex 3.4 is the other special case (p. 58). What it says: **any algebra on a product carrier is folded by a fork**.



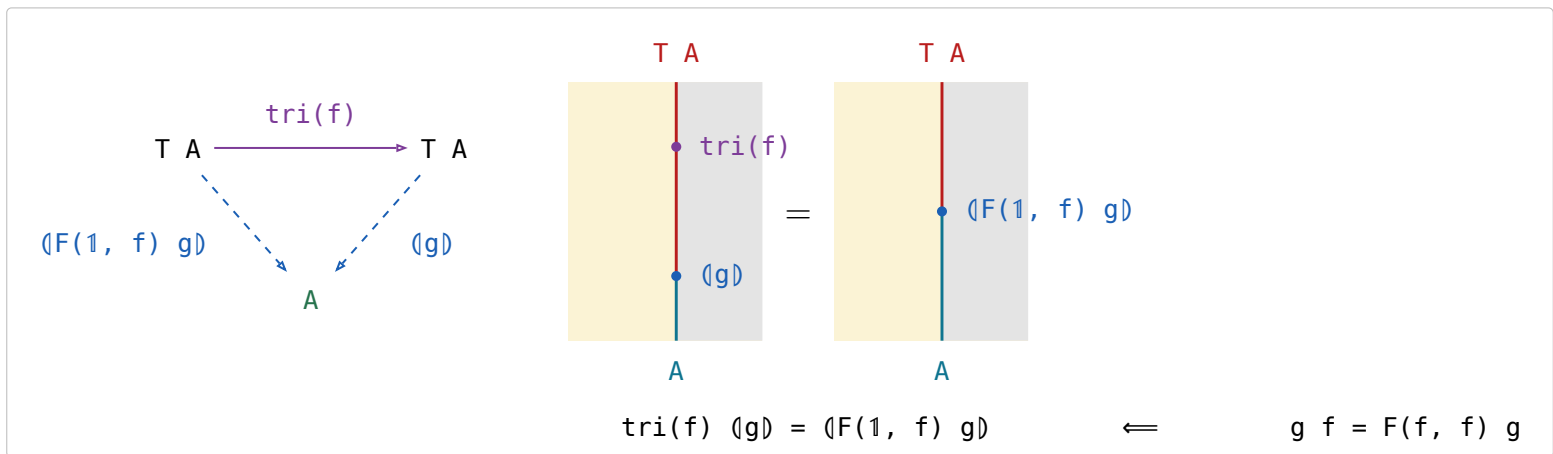
(15.5.1a)

§15.5.2. Ruby triangles

stage	definition
informally (p. 58)	$\text{tri}(f) [a_0, a_1, \dots, a_i, \dots, a_n] = [a_0, f a_1, \dots, f^i a_i, \dots, f^n a_n]$
for cons-lists (p. 59)	$\text{tri}(f) = \langle [\text{nil}, (1 \times \text{listr}(f)) \text{cons}] \rangle$
with the base functor named	$F(A, B) = 1 + A \times B, \alpha = [\text{nil}, \text{cons}], \text{ so } \text{tri}(f) = \langle F(1, \text{listr}(f)) \alpha \rangle$
abstractly (p. 59)	F a bifunctor with initial type (α, T) : $\text{tri}(f) = \langle F(1, T(f)) \alpha \rangle$

(15.5.2a)

For the definition to make sense $f : A \rightarrow A$ is required, and then $\text{tri}(f) : T A \rightarrow T A$.



(15.5.2b)

(15.5.2b) is Horner's rule, generalised from cons-lists to any initial type; B&dM call it that because for cons-lists it is the schoolbook method for evaluating a polynomial (p. 58). Fusion reduces it to $F(1, T(f)) \alpha \langle g \rangle = F(1, \langle g \rangle) F(1, f) g$ (p. 59).

§15.5.3. Depth of a tree

the datatype	$\text{tree } A ::= \text{tip } A \mid \text{bin } (\text{tree } A, \text{tree } A)$, base functor $F(A, B) = A + B \times B$, initial type $([\text{tip}, \text{bin}], \text{tree})$	(15.5.3a)
the fold	$\langle [g, h] \rangle$ is the unique f with $f (\text{tip } a) = g \ a$ and $f (\text{bin } (x, y)) = h (f \ x, f \ y)$	
$\text{tree}(f)$	$\text{tree}(f) = \langle F(f, 1) \ [\text{tip}, \text{bin}] \rangle$; pointwise $\text{tree}(f) (\text{tip } a) = \text{tip } (f \ a)$ and $\text{tree}(f) (\text{bin } (x, y)) = \text{bin } (\text{tree}(f) \ x, \text{tree}(f) \ y)$	
max	$\text{max} = \langle [1, \text{bmax}] \rangle$, where $\text{bmax } (a, b)$ is the larger of a and b	
depths	$\text{depths} = \text{tree}(\text{zero}) \ \text{tri}(\text{succ})$ — replaces every tip by its depth in the tree, zero the constant function returning 0 and succ the successor	
depth	$\text{depth} = \text{depths} \ \text{max}$	

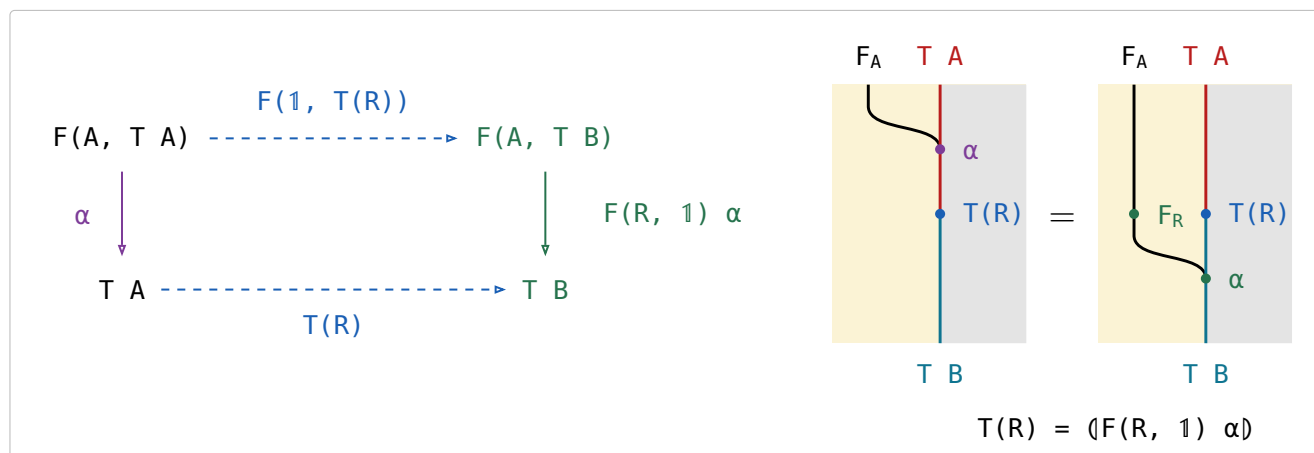
§16. Type functor

F a **binary** relator: $F(R, S)$ is its action on a pair, and $F(X)$ abbreviates $F(1, X)$, the F of the catamorphism section. For every object A the initial algebra is $\alpha : F(A, T A) \rightarrow T A$, among the maps. The **type functor** T acts on an arrow $R : A \rightarrow B$ by

$$T(R) = (F(R, 1) \alpha) : T A \rightarrow T B$$

F is the **base functor** — one layer of the structure, acting on the recursive position — while T is the datatype itself, acting on the parameter. For cons-lists, $\text{list}(R) = ([\text{nil}, (R \otimes 1) \text{cons}])$, which is $\text{map}(R)$.

A bifunctor puts its two arguments in two different places. The wire is the partial application $F_A = F(A, -)$, and an arrow in the SECOND argument is a bead on the object wire with the F wire running past it — the catamorphism section's rule, unchanged — while an arrow $R : A \rightarrow B$ in the FIRST induces a natural transformation $F_R : F_A \Rightarrow F_B$, which is a bead ON the F wire. Marsden draws bifunctors exactly so ([CatString.pdf](#), **Bifunctors**): side-by-side is already spent on functor composition, so one argument has to go into the wire's name.



$T(R)$ is the unique arrow making it commute; there is no $\sqsubseteq$ in it.

Two beads at one height on two different wires are one action, here $F(R, T(R))$, so the right panel reads $F(R, T(R)) \alpha$ and the left one $\alpha T(R)$. Their relative height says nothing: slide F_R below the $T(R)$ bead and the panel reads $F(1, T(R)) F(R, 1)$ and then α , which is the square's top row followed by its right vertical; slide it above and the panel reads $F(R, 1) F(1, T(R))$, the route through $F(B, T A)$ the square leaves out. Identifying those two is the exchange condition every bifunctor satisfies (Hinze & Marsden, Ex 1.23), and in this calculus there is nothing to identify.

name	law	what it says
the defining equation	$T(R) = (F(R, 1) \alpha)$	Rebuild the structure with α , applying R to the parameter on the way; that is $\text{map}(R)$.
functor	$T(1) = 1$ and $T(R) T(S) = T(R S)$	Mapping the identity changes nothing, and two maps in a row are one map.
type functor fusion	$T(R) (Q) = (F(R, 1) Q)$	A map followed by a fold is a single fold — the mapped structure is never built.
naturality of α	$\alpha T(R) = F(R, T(R)) \alpha$	Building and then mapping is the same as mapping the parts and then building, so α is natural from $G(R) = F(R, T(R))$ to T .
type relator	$T(R)^\circ = T(R^\circ)$	A datatype acts on relations, not only on maps — the map of the converse is the converse of the map.

Type functor fusion is the equality fusion (15.4a) applied to $T(R)$'s own defining algebra, whose side condition holds because F is a bifunctor — $F(R, 1) F(1, (Q)) = F(R, (Q)) = F(1, (Q)) F(R, 1)$ — so it too needs only the initial algebra and the defining equation.

(16d)

$$\begin{array}{ccc}
 F(A, T A) & \xrightarrow{F(R, T(R))} & F(B, T B) \\
 \downarrow \alpha & & \downarrow \alpha \\
 T A & \xrightarrow{T(R)} & T B
 \end{array}$$

It commutes strictly, and it is (16b) with the right vertical's two halves separated: push $F(R, 1)$ up into the top row, where it meets $F(1, T(R))$ as the one action $F(R, T(R))$, and α is left as the vertical. In the string diagram that is the F_R bead sliding up past $T(R)$, and what stays put is α , one bead falling past $T(R)$ from the row above to the row below — which is what “ α is natural” means.

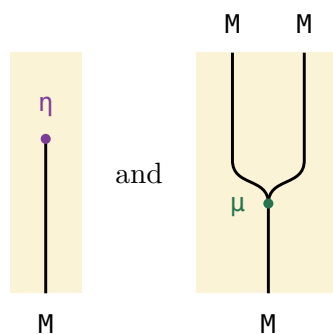
§17. Monad

(17a)

An endofunctor $M : \mathcal{A} \rightarrow \mathcal{A}$ with two natural transformations, the **unit** $\eta : \text{Id} \Rightarrow M$ and the **multiplication** $\mu : M \circ M \Rightarrow M$, subject to a unit law and an associativity law:

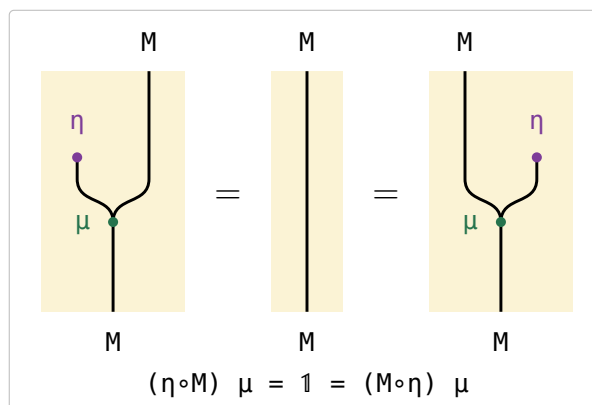
$$(\eta \circ M) \mu = 1 = (M \circ \eta) \mu \qquad (\mu \circ M) \mu = (M \circ \mu) \mu$$

(17b)



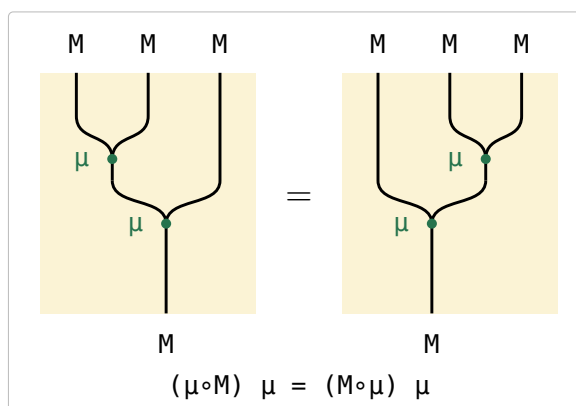
In (17b) the identity functor is a REGION and not a wire, so $\eta : \text{Id} \Rightarrow M$ has nothing above it and the M wire simply begins at its bead — the book's lollipop. $\mu : M \circ M \Rightarrow M$ is the two M wires of $M \circ M$ merging into one, its tuning fork.

(17c)



The lollipop is absorbed into the fork from either side and a bare wire is left, which is what 1 is in this calculus: nothing is drawn for it.

(17d)



The two nestings of the fork agree, so three M s collapse to one whichever pair is multiplied first.

§17.1. Monad in $\mathcal{S}et$

monad	η	μ	η on elements	μ on elements
$\text{Id } A = A$	1	1	$\eta A a = a$	$\mu A a = a$
$\text{Exc } A = A + E$	$\imath_l$	$1 \nabla \imath_r$	$\eta A a = \imath_l a$	$\mu A (\imath_l y) = y$ $\mu A (\imath_r e) = \imath_r e$
$\text{State } A = (A \times P)^P$	$\text{curry } 1$	$(\text{apply}_{A \times P})^P$	$\eta A a = \lambda x . (a, x)$	$\mu A f = \lambda x . g y \text{ where } (g, y) = f x$
$\text{Pow } A = P A$	$\{\cdot\}$	$\bigcup$	$\eta A a = \{a\}$	$\mu A Xs = \bigcup Xs$

(17.1a)

§17.2. The state monad

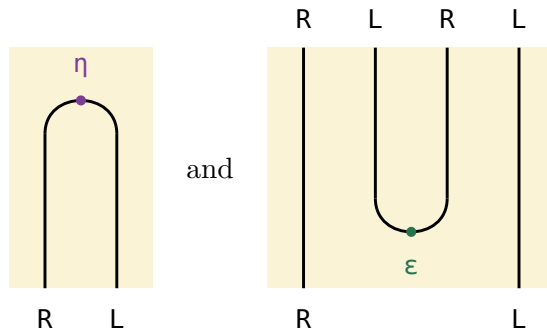
For a fixed set P of states, `IntroString p. 67` defines

$$\text{State } A = (A \times P)^P \quad \eta A = \text{curry } (1 : A \times P \rightarrow A \times P) \quad \mu A = (\text{apply}_{A \times P})^P$$

Both arrows come from the curry adjunction $L \dashv R$, $L A = A \times P$ and $R B = B^P$ (`IntroString p. 96`):

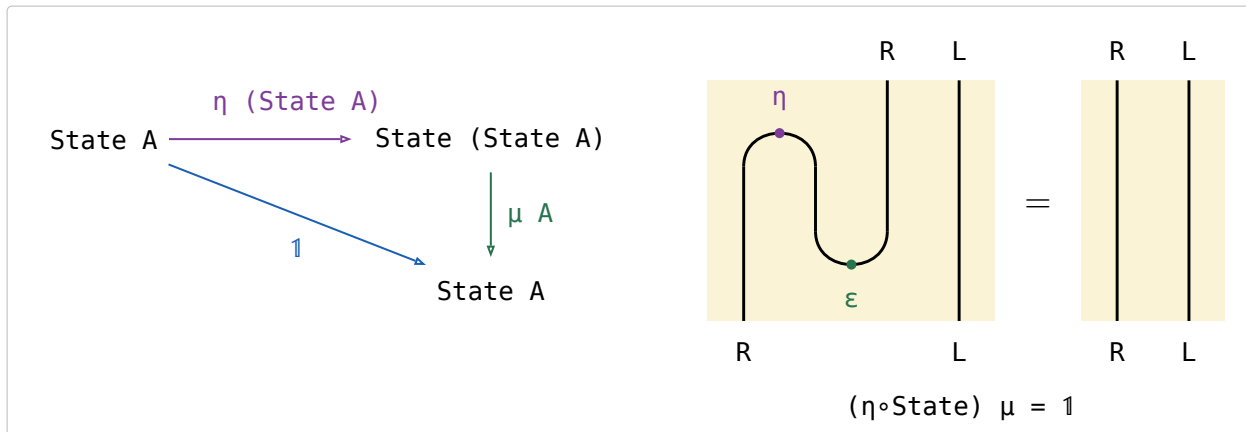
- $\text{State} = R \circ L$.
- η is the unit $\text{Id} \Rightarrow R \circ L$, which is why it is the curried identity.
- the counit $\epsilon : L \circ R \Rightarrow \text{Id}$ is `apply`.
- $\mu = R \circ \epsilon \circ L$, that counit with a functor on each side: in $\mu A = (\text{apply}_{A \times P})^P$ the subscript $A \times P$ is $L A$, the functor INSIDE, and the outer $(-)^P$ is R , the functor OUTSIDE.
- $\dashv$ composes nothing; it only names the left adjoint first.

Applicative order stands R left and L right in $\text{State} = R \circ L$. The identity functor is a region, so a unit $\text{Id} \Rightarrow R \circ L$ has nothing above it and two wires below — one strand that OPENS the pair; a counit $L \circ R \Rightarrow \text{Id}$ is that strand upside down, CLOSING a pair that arrives from above.



(17.2a)

In (17.2a), $\text{State} \circ \text{State}$ is the four wires $R L R L$, and $\mu = R \circ \epsilon \circ L$ closes the middle pair — the $L \circ R$ — leaving the outer R and L , which is State again. The two wires that run past the turn untouched are the two functors composed on, R outside and L inside.



(17.2b)

(17.2b) is the zig-zag. $\eta \circ \text{State}$ opens a new $R L$ left of the pair already there and μ closes the middle two — the new L against the old R — leaving the new R and the old L , which is State unchanged. Strip the old L , which never moves, and what zig-zags is the curry adjunction's snake equation $(\eta \circ R)(R \circ \epsilon) = 1$ (`IntroString p. 93`): (17c) read at State is that triangle identity with L composed on the inside.

In `Set` it is beta-reduction. $\eta (\text{State } A) f = \lambda x . (f, x)$ pairs the computation with the state, and μA applies it back, $\lambda x . f x$, which is f .

§18. Optimisation Problems

§18.1. Minimum and maximum

For $R : A \rightarrow A$, $\min R \triangleq \exists n (\in R) : P A \rightarrow A$, $\max R \triangleq \min R^\circ$.

$$xs (\min R) x \iff x \in xs \wedge (\forall y \in xs. y R x)$$

$$xs (\min R) x \iff (x \text{ in } xs) \text{ and all } xs R \setminus : x \text{ in } q$$

$$\min R = \exists n \text{ all } R \text{ all } R \triangleq \in R, q's \text{ all}; \text{ the chains below keep it written } \in \setminus$$

(18.1a)

the law	what it says
$X \sqsubseteq \min R \iff X \sqsubseteq \exists \text{ and } \in X \sqsubseteq R$	in the set, and below every element of it
$\{\cdot\} (\in R) = R$	bounding a singleton is bounding its element
$\Lambda(S) (\in R) = S^\circ \setminus R$	bound S 's image without building the set
$\text{union} (\in R) = \in (\in R)$ $\text{union} \triangleq \Lambda(\exists \exists) : P(P A) \rightarrow P A$	bound a union by bounding each member set
$\{\cdot\} \min R = 1 n R$	a singleton's minimum is its element, where R is reflexive $\Lambda(S) \min R \text{ at } S := 1$
$\Lambda(S) \min R = S n (S^\circ \setminus R)$	an S -value every S -value points to
$\Lambda(S) \min R = \Lambda(S) \min(R n S^\circ S)$	only R between values S gives one argument counts — context
$E(S) \min R = (\exists S) n ((\exists S)^\circ \setminus R)$	the same for the image of a set $\Lambda(S) \min R \text{ at } S := \exists S$
$P(f) \min R = \min(f R f^\circ) f$	shunt a function through a minimum
$P(S) \min R = (\exists S) n (\in (S R))$ $R \text{ reflexive}$	fusion with the power relator $\exists$ is the only proof here that tabulates
$P(S) \min R \sqsubseteq (\exists S) n (\in (S R))$	the half of the row above that costs nothing
$P(\min R) \min R \sqsubseteq \text{union} \min R$ $R \text{ a preorder}$	a minimum in each set, then a minimum of those
$P(\min R) \min R = P(\text{Dom}(\min R)) \text{ union } \min R$ $R \text{ a preorder}$	the same as an equality, once empty sets are dropped

(18.1b)

§18.1.1. $X \sqsubseteq \min R \iff X \sqsubseteq \exists \text{ and } \in X \sqsubseteq R$

(18.1.1a)

$$\boxed{\frac{X}{\min R}} \xleftrightarrow{\Delta \dashv n} \boxed{\frac{X}{\exists}} \xleftrightarrow{T \cdot \dashv T \setminus} \boxed{\frac{X}{\exists}} \xleftrightarrow{T \cdot \dashv T \setminus} \boxed{\frac{\in X}{R}}$$

§18.1.2. $\{\cdot\} (\in R) = R$

(18.1.2a)

$$\boxed{\frac{X}{\{\cdot\} (\in R)}} \xleftrightarrow{f^\circ \cdot \dashv f \cdot} \boxed{\frac{\{\cdot\}^\circ X}{\in R}} \xleftrightarrow{T \cdot \dashv T \setminus} \boxed{\frac{\in \{\cdot\}^\circ X}{R}} \xleftrightarrow{\circ} \boxed{\frac{(\{\cdot\} \exists)^\circ X}{R}} \xleftrightarrow{i \dashv E} \boxed{\frac{X}{R}}$$

§18.1.3. $\Lambda(S) (\in R) = S^\circ \setminus R$

(18.1.3a)

$$\boxed{\frac{X}{\Lambda(S) (\in R)}} \xleftrightarrow{f^\circ \cdot \dashv f \cdot} \boxed{\frac{\Lambda(S)^\circ X}{\in R}} \xleftrightarrow{T \cdot \dashv T \setminus} \boxed{\frac{\in \Lambda(S)^\circ X}{R}} \xleftrightarrow{\circ} \boxed{\frac{(\Lambda(S) \exists)^\circ X}{R}} \xleftrightarrow{\cdot \exists \dashv \Lambda} \boxed{\frac{S^\circ X}{R}} \xleftrightarrow{T \cdot \dashv T \setminus} \boxed{\frac{X}{S^\circ \setminus R}}$$

§18.1.4. $\text{union} (\in R) = \in (\in R)$

(18.1.4a)

$$\boxed{\frac{X}{\text{union} (\in R)}} \xleftrightarrow{f^\circ \cdot \dashv f \cdot} \boxed{\frac{\text{union}^\circ X}{\in R}} \xleftrightarrow{T \cdot \dashv T \setminus} \boxed{\frac{\in \text{union}^\circ X}{R}} \xleftrightarrow{\circ} \boxed{\frac{(\text{union} \exists)^\circ X}{R}} \xleftrightarrow{\cdot \exists \dashv \Lambda} \boxed{\frac{(\exists \exists)^\circ X}{R}} \xleftrightarrow{\circ, T \cdot \dashv T \setminus} \boxed{\frac{X}{\in (\in R)}}$$

§18.1.5. $\Lambda(S) \min R = S \cap (S^\circ \backslash R)$

$$\begin{array}{c}
 \boxed{\frac{X}{\Lambda(S) \min R}} \xLeftrightarrow{f^\circ \cdot \dashv f \cdot} \boxed{\frac{\Lambda(S)^\circ X}{\min R}} \xLeftrightarrow{\Delta \dashv \cap} \boxed{\frac{\Lambda(S)^\circ X}{\exists}} \xLeftrightarrow{f^\circ \cdot \dashv f \cdot} \boxed{\frac{X}{\Lambda(S) \exists}} \xLeftrightarrow{\cdot \exists \dashv \Lambda, (18.1.3a)} \boxed{\frac{X}{S}} \xLeftrightarrow{\Delta \dashv \cap} \boxed{\frac{X}{S \cap (S^\circ \backslash R)}}
 \end{array}
 \quad (18.1.5a)$$

§18.1.6. $\Lambda(S) \min R = \Lambda(S) \min (R \cap S^\circ S)$

$$\begin{array}{c}
 \boxed{\frac{X}{\Lambda(S) \min (R \cap S^\circ S)}} \xLeftrightarrow{(18.1.5a)} \boxed{\frac{X}{S \cap (S^\circ \backslash (R \cap S^\circ S))}} \xLeftrightarrow{\Delta \dashv \cap, T \cdot \dashv T \backslash} \boxed{\frac{X}{S}} \xLeftrightarrow{\Delta \dashv \cap, S^\circ \cdot \text{monotone}} \boxed{\frac{S^\circ X}{R}} \xLeftrightarrow{T \cdot \dashv T \backslash, \Delta \dashv \cap} \boxed{\frac{X}{S \cap (S^\circ \backslash R)}} \xLeftrightarrow{(18.1.5a)} \boxed{\frac{X}{\Lambda(S) \min R}}
 \end{array}
 \quad (18.1.6a)$$

§18.1.7. $P(f) \min R = \min(f R f^\circ) f$

$$\begin{array}{c}
 \boxed{\frac{P(f) \min R}{f \text{ a map}}} \xLeftrightarrow{P = E \text{ on maps, (18.1.5a)}} \boxed{(\exists f) \cap ((\exists f)^\circ \backslash R)} \xLeftrightarrow{\cdot^\circ, T \cdot \dashv T \backslash, f^\circ \cdot \dashv f \cdot} \boxed{(\exists f) \cap (E \backslash (f R))} \\
 \xLeftrightarrow{\text{modular law, } f \text{ simple}} \boxed{(\exists \cap ((E \backslash (f R)) f^\circ)) f} \xLeftrightarrow{\cdot f \dashv \cdot f^\circ, \min} \boxed{\min(f R f^\circ) f}
 \end{array}
 \quad (18.1.7a)$$

§18.1.8. $P(S) \min R = (\exists S) \cap (E \backslash (S R))$

(p, q) tabulates $W \triangleq (\exists S) \cap (E \backslash (S R))$, and $y \triangleq \Lambda((p \exists S) \cap (q R^\circ))$, a map.

$$\boxed{\frac{W}{P(S) \min R}} \xLeftrightarrow{p^\circ q = W, 1 \sqsubseteq y y^\circ} \boxed{\frac{p^\circ y}{P(S)}} \quad \boxed{\frac{y^\circ q}{\min R}}
 \quad (18.1.8b)$$

$$\boxed{\frac{y^\circ q}{\exists}} \xLeftrightarrow{f^\circ \cdot \dashv f \cdot, \cdot \exists \dashv \Lambda} \boxed{\frac{q}{(p \exists S) \cap (q R^\circ)}} \xLeftrightarrow{\Delta \dashv \cap, f^\circ \cdot \dashv f \cdot} \boxed{\frac{p^\circ q}{\exists S}} \quad \boxed{\frac{1}{R^\circ}} \quad R \text{ reflexive}
 \quad (18.1.8c)$$

$$\boxed{\frac{E y^\circ q}{R}} \xLeftrightarrow{\cdot \exists \dashv \Lambda, \cdot^\circ, \text{meet}} \boxed{\frac{R q^\circ q}{R}} \quad q \text{ simple}$$

$$\boxed{\frac{p^\circ y}{P(S)}} \xLeftrightarrow{\Delta \dashv \cap, \cdot T \dashv T, \cdot^\circ, f^\circ \cdot \dashv f \cdot} \boxed{\frac{p^\circ y \exists}{\exists S}} \quad \boxed{\frac{p \exists}{y \exists S^\circ}}
 \quad (18.1.8d)$$

$$\boxed{\frac{p^\circ y \exists}{\exists S}} \xLeftrightarrow{\cdot \exists \dashv \Lambda, \text{meet}} \boxed{\frac{p^\circ p \exists S}{\exists S}} \quad p \text{ simple}$$

$$\boxed{\frac{p \exists}{y \exists S^\circ}} \xLeftrightarrow{\text{modular law}} \boxed{\frac{p \exists}{(p \exists) \cap (q R^\circ S^\circ)}} \xLeftrightarrow{1 \sqsubseteq q q^\circ} \boxed{\frac{q^\circ p \exists}{R^\circ S^\circ}} \xLeftrightarrow{\cdot^\circ, T \cdot \dashv T \backslash} \boxed{\frac{W}{E \backslash (S R)}} \quad W \text{'s right half}$$

§18.1.9. $P(S) \min R \sqsubseteq (\exists S) \cap (\epsilon \backslash (S R))$

(18.1.9a)

$$\begin{array}{ccccc}
 \boxed{\frac{P(S) \min R}{(\exists S) \cap (\epsilon \backslash (S R))}} & \xleftarrow{\Delta \dashv \cap, T \cdot \dashv T \backslash} & \boxed{\frac{P(S) \min R}{\exists S}} & \xleftarrow{\text{UP of min}} & \boxed{\frac{P(S) \exists}{\exists S}} \\
 & & \boxed{\frac{\epsilon P(S) \min R}{S R}} & & \boxed{\frac{\epsilon P(S)}{S \epsilon}}
 \end{array}$$

§18.1.10. $P(\min R) \min R \sqsubseteq \text{union min R}$

(18.1.10a)

$$\begin{array}{ccccc}
 \boxed{\frac{P(\min R) \min R}{\text{union min R}}} & \xleftrightarrow{(18.1.5a)} & \boxed{\frac{P(\min R) \min R}{(\exists \exists) \cap ((\exists \exists)^\circ \backslash R)}} & \xleftarrow{\circ, \Delta \dashv \cap, T \cdot \dashv T \backslash} & \boxed{\frac{P(\min R) \min R}{\exists \exists}} \\
 \text{R a preorder} & & & & \xleftarrow{\text{UP of min, R transitive}} \boxed{\frac{P(\min R) \exists}{\exists \min R}} \\
 & & & & \boxed{\frac{\epsilon P(\min R)}{\min R \epsilon}}
 \end{array}$$

§18.1.11. $P(\min R) \min R = P(\text{Dom}(\min R)) \text{union min R}$

(18.1.11a)

$$\begin{array}{ccc}
 \boxed{P(\min R) \min R} & \xrightarrow{\text{Dom}(\min R) \min R = \min R} & \boxed{P(\text{Dom}(\min R) \min R) \min R} \\
 \xrightarrow{\text{P a relator}} & \boxed{P(\text{Dom}(\min R)) P(\min R) \min R} & \xrightarrow{(18.1.10a)} \boxed{P(\text{Dom}(\min R)) \text{union min R}}
 \end{array}$$

(18.1.11b)

$$\begin{array}{ccc}
 \boxed{\frac{P(\text{Dom}(\min R)) \text{union min R}}{P(\min R) \min R}} & \xleftarrow{(18.1.8b) \text{ at } S := \min R, \Delta \dashv \cap, T \cdot \dashv T \backslash} & \boxed{\frac{P(\text{Dom}(\min R)) \text{union min R}}{\exists \min R}} \\
 & & \boxed{\frac{\epsilon P(\text{Dom}(\min R)) \text{union min R}}{\min R R}}
 \end{array}$$

$$\begin{array}{ccc}
 \boxed{\frac{P(\text{Dom}(\min R)) \text{union min R}}{\exists \min R}} & \xleftarrow{P(\text{Dom}(\min R)) \sqsubseteq 1} & \boxed{\frac{\text{union min R}}{\exists \min R}}
 \end{array}$$

$$\begin{array}{ccc}
 \xleftarrow{T(U \cap V) \sqsubseteq T U \cap T V, \cdot \exists \dashv \cap, (18.1.4a)} & \boxed{\frac{(\exists \exists) \cap (\epsilon \backslash (\epsilon \backslash R))}{\exists \min R}} & \xleftarrow{\text{modular law}} \boxed{\frac{\exists (\exists \cap (\epsilon \backslash (\epsilon \backslash R)))}{\exists (\exists \cap (\epsilon \backslash R))}} \\
 & & \text{counit of } T \cdot \dashv T \backslash
 \end{array}$$

$$\begin{array}{ccc}
 \boxed{\frac{\epsilon P(\text{Dom}(\min R)) \text{union min R}}{\min R R}} & \xleftarrow{\epsilon \text{ lax natural}} & \boxed{\frac{\text{Dom}(\min R) \epsilon \text{union min R}}{\min R R}}
 \end{array}$$

$$\begin{array}{ccc}
 \xleftarrow{\text{Dom}(\min R) \sqsubseteq \min R (\min R)^\circ} & \boxed{\frac{\min R (\min R)^\circ \epsilon \text{union min R}}{\min R R}} & \xleftarrow{(\min R)^\circ \sqsubseteq \epsilon, \epsilon \epsilon \text{union} \sqsubseteq \epsilon} \boxed{\frac{\min R \epsilon \min R}{\min R R}} \\
 & & \text{UP of min}
 \end{array}$$

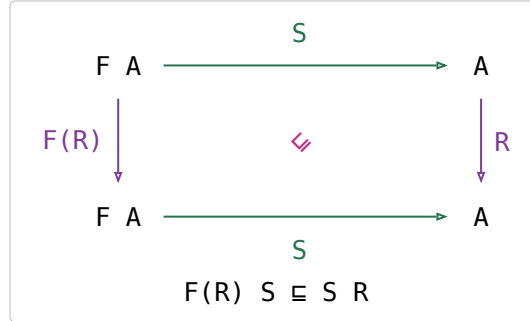
§18.2. Monotonic algebras

An F -algebra $S : F A \rightarrow A$ is **monotonic on** $R : A \rightarrow A$ if $F(R) S \sqsubseteq S R$.

(18.2a)

For a map $f : F A \rightarrow A$ that is $f \circ F(R) f \sqsubseteq R$, equivalently $F(R) \sqsubseteq f R f \circ$.

$(\text{leq} \times \text{leq})$ plus $\sqsubseteq$ plus leq — addition on Nat is monotonic on leq , which at the point level reads $c = a + b \wedge a \leq a' \wedge b \leq b' \implies c \leq a' + b'$.



(18.2b)

$f : F A \rightarrow A$ **distributes over** R if $F(\min R) f \sqsubseteq \Lambda(F(\exists) f) \min R$.

(18.2c)

plus distributes over leq , at the point level $\min x + \min y = \min\{a + b \mid a \in x \wedge b \in y\}$ for x, y non-empty and $\min \triangleq \min \text{leq}$.

the law	what it says	(18.2d)
$f \circ F(R) f \sqsubseteq R \iff f \circ F(R^\circ) f \sqsubseteq R^\circ$	a map is monotonic on an order and on its opposite together — a relation is not	
$f \circ F(R) f \sqsubseteq R \iff F(\min R) f \sqsubseteq \Lambda(F(\exists) f) \min R$ Theorem 7.1, f a map	for a map the two definitions above are one condition	
$(\Lambda(S) \min R) \sqsubseteq \Lambda((S)) \min R$ Theorem 7.2, R° a preorder and $F(R^\circ) S \sqsubseteq S R^\circ$	the greedy theorem : keeping one minimum at every step beats no more than taking the minimum at the end	

§18.3. Planning a company party

tree $A ::= \text{node}(A, \text{list}(\text{tree } A))$, base functor $F(A, B) = A \times \text{list } B$, rating : $\text{Employee} \rightarrow \text{Real}$.

(18.3a)

cost $\triangleq \text{list}(\text{rating}) \text{ sum}$, $R \triangleq \text{cost} \text{ leq } \text{cost}^\circ$, choose $\triangleq \text{outl} \cup \text{outr}$.

include $\triangleq (1 \times (\text{list}(\text{outr}) \text{ concat})) \text{ cons}$ exclude $\triangleq (1 \times (\text{list}(\text{choose}) \text{ concat})) \text{ outr}$

$S \triangleq \{\text{include}, \text{exclude}\}$, party $\triangleq (S) \text{ choose} : \text{tree } \text{Employee} \rightarrow \text{list } \text{Employee}$

the law	what it says	(18.3b)
$\Lambda(\text{party}) \max R$	the specification: a guest list of greatest total conviviality	
$(R \times R) \text{ choose} \sqsubseteq \text{choose } R$	choose is monotonic on R — the first claim	
$(1 \times \text{list}(R \times R)) S \sqsubseteq S (R \times R)$	S is monotonic on $R \times R$ — the second claim	
$\Lambda(\text{party}) \max R \sqsupseteq (\Lambda(S) \max(R \times R)) \Lambda(\text{choose}) \max R$	the greedy theorem: one best pair per subtree instead of all parties of the whole tree	
party = $(S) \Lambda(\text{choose}) \max R$ include = $(1 \times (\text{list}(\text{outr}) \text{ concat})) \text{ cons}$ exclude = $\text{outr list}(\Lambda(\text{choose}) \max R) \text{ concat}$	the program, exclude renamed: include was already a map, and exclude becomes one	

§18.4. Shortest paths on a cylinder

$F(A, X) = A + A \times X$, $L = \text{list}^+$ with initial algebra $\alpha : F(A, L A) \rightarrow L A$, N the n -tuple relator.

(18.4a)

$R \triangleq \text{sum} \text{ leq } \text{sum}^\circ$, setify : $N A \rightarrow P A$, moves : $N A \rightarrow P N A$, trans : $P N A \rightarrow N P A$, zip : $F(N A, N B) \rightarrow N F(A, B)$, cp $\triangleq \Lambda(F(1, \exists))$.


```

generate  $\triangleq$  F(1, moves trans N(union)) zip N(cp P( $\alpha$ )) : F(N A, N P L A)  $\rightarrow$  N P L A
paths  $\triangleq$  (generate) setify union : L N Nat  $\rightarrow$  P L Nat

```

the law	what it says	(18.4b)
paths min R	the specification: a cheapest path across the cylinder	
F(1, min R) $\alpha \sqsubseteq$ cp P(α) min R (7.13), Theorem 7.1 at the map α , using $\Lambda(F(1, \exists) \alpha) = \text{cp } P(\alpha)$	extending every path in a set and then taking a minimum is beaten by extending one minimum	
generate N(min R) $\sqsupseteq$ F(1, N(min R)) Q	the fusion condition that defines Q — this is what (7.13) is spent on	
Q = F(1, moves trans N(min R)) zip N(α) = [N(wrap), (1 $\times$ moves trans N(min R)) zip' N(cons)]	generate with the minimum taken inside each tuple component	
paths min R $\sqsupseteq$ (Q) setify min R	the program: one fold, n best paths carried per column	

§18.5. The security van problem

R $\triangleq$ length leq length $^\circ$, ceiling $\triangleq$ $\Lambda(\text{prefix sum}) \text{ max leq}$, floor $\triangleq$ $\Lambda(\text{prefix sum}) \text{ min leq}$.

secure the coreflexive on x with bmax(ceiling x, ceiling x - floor x) $\leq$ N; ok the coreflexive on (a, xs) with xs non-empty and [a] $\#$ head xs secure.

new $\triangleq$ (wrap $\times$ 1) cons glue $\triangleq$ (1 $\times$ cons $^\circ$) assocl (cons $\times$ 1) cons

old $\triangleq$ (1 $\times$ cons $^\circ$) assocl ((cons secure) $\times$ 1) cons

partition = ([nil, new u glue]), S $\triangleq$ [nil, new u old], partition list(secure) = (S).

H $\triangleq$ (head prefix head $^\circ$) u (nil $^\circ$ nil), R ; H $\triangleq$ R $\cap$ (R $^\circ \Rightarrow$ H), |R| $\triangleq$ R $\cap$ $\neg$ R $^\circ$.

the law	what it says	(18.5a)
$\Lambda(\text{partition list(secure)}) \text{ min R}$	the specification: fewest secure segments	
secure prefix $\sqsubseteq$ prefix secure	secure is prefix-closed — the only property of it used until the program	
(1 $\times$ R $^\circ$) new $\sqsubseteq$ (new u old) R $^\circ$ (7.14)	starting a new segment is monotonic on R $^\circ$	
(1 $\times$ R $^\circ$) old $\sqsubseteq$ (new u old) R $^\circ$ (7.15), FALSE	extending the first segment is not — the shorter partition need not stay secure	
(1 $\times$ (R ; H) $^\circ$) new $\sqsubseteq$ (new u old) (R ; H) $^\circ$ (7.16) (1 $\times$ (R ; H) $^\circ$) old $\sqsubseteq$ (new u old) (R ; H) $^\circ$ (7.17)	both halves do become monotonic once R is refined by H, which breaks a tie in length by the first segment's prefix order	
(1 $\times$ T) new $\sqsubseteq$ new H $^\circ$ (7.18)	whatever the tail, new's first segment is [a], a prefix of every first segment — this is what (7.16) rests on	
(1 $\times$ T) old $\sqsubseteq$ new H $^\circ$ (7.19) (1 $\times$ R $^\circ$) old $\sqsubseteq$ new R $^\circ$ (7.20) (1 $\times$ (R $^\circ \cap$ H $^\circ$)) old $\sqsubseteq$ old (R $^\circ \cap$ H $^\circ$) (7.21)	the three claims (7.17) rests on, split along (R ; H) $^\circ$ = R $^\circ$ u (R $^\circ \cap$ H $^\circ$)	
old $\sqsubseteq$ new (R ; H)	old returns a shorter result than new if it returns any result at all, so the fold's choice refines to a map	
([nil, (ok $\rightarrow$ glue, new)]) $\sqsubseteq$ ($\Lambda(S) \text{ min}(R ; H)$)	the program: glue onto the first segment while it stays secure, else start a new one	

§19. Thinning Algorithms

§19.1. Thinning

For $Q : A \rightarrow A$, $\text{thin } Q \triangleq (\exists/\exists) \cap (\in \backslash (Q \in)) : P A \rightarrow P A$ (8.1).

$$y (\text{thin } Q) x \iff x \subseteq y \wedge (\forall a \in y. \exists b \in x. a Q b)$$

(19.1a)

the law	what it says	(19.1b)
$X \subseteq \Lambda(S) \text{ thin } Q \iff X \ni \subseteq S \text{ and } S^\circ X \subseteq Q \in$	the universal property: everything kept is an S -value, and every S -value has a Q -lower bound among the kept ones	
$Q \subseteq R \implies \text{thin } Q \subseteq \text{thin } R$	the fewer pairs Q relates, the fewer subsets count as thinnings	
$1 \subseteq \text{thin } Q$, and $\text{thin } Q$ is a preorder if Q is	keeping everything is always a legal thinning	
$\min R = \text{thin } Q \min R$ $Q \subseteq R$, both preorders	thin-introduction : thinning first cannot lose an R -minimum	
$\text{thin } Q \supseteq \min Q \{\cdot\}$ (8.2)	thin-elimination : keeping one element is a thinning, but its domain is the sets $\min Q$ is defined on	
$\Lambda(S) \text{ thin } Q \supseteq \Lambda(S) \min R \{\cdot\}$ (8.3), $R \cap (S^\circ S) \subseteq Q$	the usable variant: R need only refine Q between values S gives one argument	
$\text{union thin } Q \supseteq P(\text{thin } Q) \text{ union}$ (8.4)	thinning each member set is a thinning of the union	
$(\Lambda(F(\exists) S) \text{ thin } Q) \subseteq \Lambda((\exists S)) \text{ thin } Q$ Theorem 8.1, S monotonic on Q°	the thinning theorem : thinning at every step beats thinning only at the end	
$(\Lambda(F(\exists) S) \text{ thin } Q) \min R \subseteq \Lambda((\exists S)) \min R$ Corollary 8.1, $Q \subseteq R$ as well	the same against the optimisation problem itself, by thin-introduction	

§19.2. Paths in a layered network

$F(A, X) = A + A \times X$, $L = \text{list}^+$ with initial algebra $\alpha \triangleq [\text{wrap}, \text{cons}] : F(A, L A) \rightarrow L A$.

$\text{wrapz} \triangleq \langle \text{wrap}, \text{zero} \rangle$, $\text{consw } (a, (x, n)) = (\text{cons } (a, x), \text{wt } (a, \text{head } x) + n)$.

$\text{cost} \triangleq ([\text{wrapz}, \text{consw}]) \text{ outr}$, $([\text{wrapz}, \text{consw}]) = \langle 1, \text{cost} \rangle$, $R \triangleq \text{cost} \text{ leq } \text{cost}^\circ$.

$Q \triangleq R \cap (\text{head } \text{head}^\circ)$, $S \triangleq F(1, \exists) \alpha$, $\Lambda(F(\exists, 1)) = 1 + \text{cpl}$, $\Lambda(F(1, \exists)) = 1 + \text{cpr}$, $\text{step} \triangleq \text{cpr } P(\text{cons}) \min R$.

(19.2a)

the law	what it says	(19.2b)
$\Lambda((F(\exists, 1) \alpha)) \min R = \Lambda(L(\exists)) \min R$	the specification: a least-cost path across the network	
$F(\exists, Q^\circ) \alpha \subseteq F(\exists, 1) \alpha Q^\circ$	S is monotonic on Q° ; on R° it is not, since the next edge can cost arbitrarily much	
$(\Lambda(F(\exists, \exists) \alpha) \text{ thin } Q) \min R \subseteq \Lambda((F(\exists, 1) \alpha)) \min R$ Corollary 8.1	thinning applies: one partial path per starting vertex is enough	
$S \text{ head} \subseteq [1, \text{outr}]$ $S \text{ head}$ simple, so $R \cap (S^\circ S) \subseteq Q$	the side condition of (8.3): between two paths S builds from one argument, equal cost and equal head already means Q	
$\Lambda(F(\exists, \exists) \alpha) \text{ thin } Q$ $\supseteq \Lambda(F(\exists, 1)) P(\Lambda(F(1, \exists))) P(\alpha) \min R$ (8.4), (8.3), $P(\{\cdot\}) \text{ union} = 1$	thin eliminated: split the algebra at $F(\exists, 1) F(1, \exists)$, thin each part, one minimum per part	
$\Lambda(F(1, \exists)) P(\alpha) \min R = [\text{wrap}, \text{step}]$	that inner part read off the two branches of α	
$\Lambda((F(\exists, 1) \alpha)) \min R \supseteq ([P(\text{wrap}), \text{cpl } P(\text{step})]) \min R$	the program: one fold, a best path per vertex of the current layer	

§19.3. Implementing thin

(19.3a)

$\text{setify} : \text{list } A \rightarrow P \ A$, $\text{cup} : P \ A \times P \ A \rightarrow P \ A$, $\text{cp}(F) \triangleq \Lambda(F(\exists))$, $\text{listcp}(F) : F(\text{list } A) \rightarrow \text{list}(F \ A)$, $\text{sort } P \triangleq \text{setify}^\circ \text{ ordered } P$ for P a connected preorder.

$\text{thinlist } Q$ is any $\text{thinlist } Q \sqsubseteq \text{subseq}$ with $\text{thinlist } Q \text{ setify} \sqsubseteq \text{setify thin } Q$; one is $\langle [\text{nil}, \text{bump } Q] \rangle$, $\text{bump } Q \ (a, []) = [a]$, $\text{bump } Q \ (a, [b] \# x) = (b \ Q \ a \rightarrow [a] \# x, a \ Q \ b \rightarrow [b] \# x, [a] \# [b] \# x)$.

Binary thinning data: $S = (f_1 \ p_1) \cup (f_2 \ p_2)$ with p_1, p_2 coreflexive; Q a preorder with $Q \sqsubseteq R$ and both $f_1 \ p_1, f_2 \ p_2$ monotonic on Q° ; P a connected preorder with both f_1, f_2 monotonic on P ; $g_i \triangleq \text{list}(f_i) \text{ filter}(p_i)$.

the law	what it says	(19.3b)
$\text{thinlist } Q \ x = [\text{minlist } Q \ x]$ (8.5), Q connected, x non-empty	what thinning should come to when it can: one element	
$\text{sort } P \ \text{thinlist } Q \sqsubseteq \text{thin } Q \ \text{sort } P$ (8.6)	thinning a sorted list is a thinning of the set — this is what $\text{thinlist } Q \sqsubseteq \text{subseq}$ buys	
$\text{sort } P \ \text{minlist } Q \sqsubseteq \text{min } Q$ (8.7)	a minimum of the sorted list is a minimum of the set	
$\text{sort}(f \ P \ f^\circ) \ \text{list}(f) \sqsubseteq P(f) \ \text{sort } P$ (8.8)	shunt a function through a sort	
$\text{sort } P \ \text{filter}(p) \sqsubseteq E(p) \ \text{sort } P$ (8.9), p coreflexive	filtering a sorted list sorts the restricted set	
$(\text{sort } P \times \text{sort } P) \ \text{merge } P \sqsubseteq \text{cup sort } P$ (8.10)	merging two sorted lists sorts their union	
$F(\text{sort } P) \ \text{listcp}(F) \sqsubseteq \text{cp}(F) \ \text{sort}(F \ P)$ (8.11), F linear	$\text{listcp}(F)$ is the list implementation of the cartesian product $\text{cp}(F)$	
$F(\text{sort } P) \ \text{listcp}(F) \ \text{list}(f) \ \text{filter}(p) \sqsubseteq \Lambda(F(\exists) \ f \ p) \ \text{sort } P$ Lemma 8.1, f monotonic on P , p coreflexive	one sorted list built from sorted arguments, instead of a set built and then sorted	
$\langle \text{listcp}(F) \ \langle g_1, g_2 \rangle \ \text{merge } P \ \text{thinlist } Q \rangle \ \text{minlist } R \sqsubseteq \Lambda(\langle S \rangle) \ \text{min } R$ Theorem 8.2	the binary thinning theorem : a fold on sorted lists of partial solutions, thinned at every step	

§19.4. The knapsack problem

(19.4a)

$\text{vol}, \text{wt} : \text{Item} \rightarrow \text{Real}$, $\text{value} \triangleq \text{list}(\text{vol}) \ \text{sum}$, $\text{weight} \triangleq \text{list}(\text{wt}) \ \text{sum}$.

$\text{subseq} = \langle [\text{nil}, \text{cons}] \cup [\text{nil}, \text{outr}] \rangle$, within w the coreflexive on x with $\text{weight } x \leq w$.

$\text{geq} \triangleq \text{leq}^\circ$, $R \triangleq \text{value geq value}^\circ$, $Q \triangleq R \cap (\text{weight leq weight}^\circ)$, $P \triangleq R$.

$F \ A = 1 + \text{Item} \times A$, $\text{listcp}(F) = \text{wrap} + \text{cpr}$, $g_1 \triangleq \text{list}([\text{nil}, \text{cons}]) \ \text{filter}(\text{within } w) = [\text{list}(\text{nil}), h_1]$, $g_2 \triangleq \text{list}([\text{nil}, \text{outr}]) = [\text{list}(\text{nil}), h_2]$.

$h_1 \triangleq \text{list}(\text{cons}) \ \text{filter}(\text{within } w)$, $h_2 \triangleq \text{list}(\text{outr})$.

the law	what it says	(19.4b)
$\Lambda(\text{subseq} \ (\text{within } w)) \ \text{min } R$	the specification: a selection of greatest total value that stays within the capacity w	
$\text{subseq} \ (\text{within } w) = \langle ([\text{nil}, \text{cons}] \ (\text{within } w)) \cup [\text{nil}, \text{outr}] \rangle$ fusion, weights non-negative	the selections that fit are themselves a fold, and the fold's algebra already has binary thinning's shape	
$(1 \times R^\circ) \ (\text{cons} \ (\text{within } w)) \sqsubseteq \text{cons} \ (\text{within } w) \ R^\circ$ FALSE	not monotonic on R° : a selection of greater value need not still fit once one more item goes in	
$(1 \times Q^\circ) \ (\text{cons} \ (\text{within } w)) \sqsubseteq \text{cons} \ (\text{within } w) \ Q^\circ$ $(1 \times Q^\circ) \ \text{outr} \sqsubseteq \text{outr } Q^\circ$	both halves are monotonic on Q° once ties in value are broken by weight	
$\langle \text{listcp}(F) \ \langle g_1, g_2 \rangle \ \text{merge } R \ \text{thinlist } Q \rangle \ \text{minlist } R$ Theorem 8.2 at $P \triangleq R$, F linear	binary thinning, sorting in descending order of value	
$\text{knapsack } w = \langle [\text{nil}, \text{cpr} \ \langle h_1, h_2 \rangle \ \text{merge } R \ \text{thinlist } Q] \rangle \ \text{minlist } R$	the program; $\text{minlist } R$ becomes head , since packings come out in descending value	

§19.5. The paragraph problem

(19.5a)

```

Line = list+ Word, Para = list+ Line, F A = Word + Word × A, listcp(F) = wrap + cpr.

new (a, xs) = [[a]] # xs, glue (a, xs) = [[a] # head xs] # tail xs, partition ≐ ([wrap wrap, new u
glue]) : list+ Word → Para.

width ≐ ([length, (length × 1) plus succ]), fits w the coreflexive on a line x with width x ≤ w, ok w the
coreflexive on [x] # xs with width x ≤ w.

white w x = w - width x, collect ≐ list(sqr) sum, waste w ≐ init list(white w) collect.

R ≐ (waste w) leq (waste w)o, Q ≐ R n (head heado), P ≐ T.

g1 ≐ list([wrap wrap, new]), g2 ≐ list([wrap wrap, glue]) filter(ok w), start ≐ wrap wrap wrap, h1 ≐
list(new), h2 ≐ list(glue) filter(ok w).

```

the law	what it says	(19.5b)
$\Lambda(\text{partition list}^+(\text{fits } w)) \min R$	the specification: least waste among the paragraphs whose every line fits	
$\text{partition list}^+(\text{fits } w) = ([\text{wrap wrap}, \text{new } u (\text{glue } (\text{ok } w))])$ fusion, every word fits on a line by itself	the fitting paragraphs are themselves a fold	
$[\text{wrap wrap}, \text{new } u (\text{glue } (\text{ok } w))]$ $= [\text{wrap wrap}, \text{new}] \cup ([\text{wrap wrap}, \text{glue}] (\text{ok } w))$	rewritten into binary thinning's shape $(f_1 \ p_1) \cup (f_2 \ p_2)$	
$(1 \times R^o) \text{ glue } \sqsubseteq \text{ glue } R^o \quad \text{FALSE}$	glue is not monotonic on R^o : the waste of a paragraph depends on its whole first line, so no greedy algorithm solves this	
$(1 \times Q^o) \text{ new } \sqsubseteq \text{ new } Q^o$ $(1 \times Q^o) (\text{glue } (\text{ok } w)) \sqsubseteq \text{ glue } (\text{ok } w) Q^o$ cons monotonic on collect leq collect ^o	both halves are monotonic on Q^o once ties in waste are broken by the first line	
merge $T = \text{cat}$; $P \triangleq \text{head prefix head}^o$ also serves	T needs no sorting at all, and prefix is a linear order on first lines of paragraphs of one input	
$([\text{listcp}(F) (g_1, g_2) \text{ cat thinlist } Q]) \minlist R$ Theorem 8.2 at $P \triangleq T$	binary thinning, one candidate kept per first line	
paragraph $w = ([\text{start}, \text{cpr } (h_1, h_2) \text{ cat thinlist } Q]) \minlist R$	the program, g_1 and g_2 split along the coproduct	

§19.6. Bitonic tours

(19.6a)

```

F A = (City × City) + (City × A), the base functor of cons-lists of length at least two; listcp(F) = wrap + cpr;
tc : City × City → Real, neither positive nor symmetric.

start (a, b) = ([a, b], [a, b]), dropl (a, ([b] # x, y)) = ([a] # x, [a] # y), dropr (a, (x, [b] # y))
= ([a] # x, [a] # y), tour ≐ ([start, dropl u dropr]).

cost (x, y) = outcost x + incost y, outcost [a0, ..., an] = tc (a0, a1) + ... + tc (an-1, an), incost [a0,
..., an] = tc (a1, a0) + ... + tc (an, an-1).

next ≐ tail head, next2 ≐ next × next, R ≐ cost leq costo, Q ≐ R n (next2 next2o), P ≐ T, g1 ≐
list([start, dropl]), g2 ≐ list([start, dropr]).

```

the law	what it says	(19.6b)
$\Lambda(\text{tour}) \min R$	the specification: a least-cost bitonic tour, outward journey and return kept as a pair of lists	
$(1 \times R^\circ) \text{ dropl} \sqsubseteq \text{dropl } R^\circ \quad \text{FALSE}$ $(1 \times R^\circ) \text{ dropr} \sqsubseteq \text{dropr } R^\circ \quad \text{FALSE}$	neither drop is monotonic on R° : the two edges it adds and removes depend on head and next of both lists	
$(1 \times Q^\circ) \text{ dropl} \sqsubseteq \text{dropl } Q^\circ$ $(1 \times Q^\circ) \text{ dropr} \sqsubseteq \text{dropr } Q^\circ$	both are, once ties in cost are broken by the two second cities — the heads already agree among tours of one input	
$(\text{listcp}(F) \langle g_1, g_2 \rangle \text{ cat thinlist } Q) \minlist R$ Theorem 8.2 at $P \triangleq T$, merge $T = \text{cat}$	binary thinning, one candidate per pair of second cities	
$\text{mintour} = ([\text{start wrap}, \text{cpr } (\text{list}(\text{dropl}), \text{list}(\text{dropr})) \text{ cat thinlist } Q]) \minlist R$	the program: quadratic, because each step adds just two tours to the list kept	

§20. Dynamic Programming

§20.1. Theory

$h : F B \rightarrow B$ a map, $T : F A \rightarrow A$ an F -algebra, $R : B \rightarrow B$.

(20.1a)

$H \triangleq (T)^\circ (h) : A \rightarrow B$, $M \triangleq \Lambda(H) \min R$ the problem to be solved, $(\mu X : G(X))$ the least fixed point of G .

the law	what it says	(20.1b)
$(\mu X : \Lambda(T^\circ) P(F(X) h) \min R) \sqsubseteq M$ Theorem 9.1, h monotonic on R	dynamic programming: decompose in every way, solve each part, keep one optimum per part	
$\Lambda(T^\circ) P(F(M) h) \min R \sqsubseteq M$ (9.1)	all Knaster–Tarski leaves to prove	
$\Lambda(T^\circ) P(F(M) h) \min R \sqsubseteq H$ (9.2) $H^\circ \Lambda(T^\circ) P(F(M) h) \min R \sqsubseteq R$ (9.3)	(9.1) split by the universal property of $\min$	
$P(X) \min R \sqsubseteq (\exists X) \cap (E \setminus (X R))$ (9.4) = (7.10), (18.1b)	the only fact about $\min$ either proof uses	
$(\mu X : \Lambda(T^\circ) \text{thin } Q P(F(X) h) \min R) \sqsubseteq M$ Theorem 9.2, Q a preorder with $Q^\circ F(H) h \sqsubseteq F(H) h R^\circ$; $\text{thin } Q$ as in (19.1b)	the same with a thinning step: decompositions that can never win are dropped	
the fixed point is unique, and entire Theorem 6.3, T° followed by F 's membership relation inductive; $\Lambda(T^\circ)$ finite and non-empty, R connected	when the recursion can be refined to a recursive function	
$\Lambda([V_1, V_2]^\circ) \text{thin}(Q_1 + Q_2) P([U_1, U_2]) \min R$ $= (\text{Ran } V_1 \rightarrow W_1, W_2)$ $W_i \triangleq \Lambda(V_i^\circ) \text{thin}(Q_i) P(U_i) \min R$ Proposition 9.1, $V_2 V_1^\circ = \perp$	$F A$ is usually a coproduct, and disjoint ranges split the fixed point into one branch per summand	
$F(R) h \sqsubseteq h R$ $Q^\circ F(H) h \sqsubseteq F(H) h R^\circ$	the two conditions to be checked, in the order the three propositions below discharge them	
$F(R) h \sqsubseteq h R$ Proposition 9.2, $R \triangleq \text{cost} \text{ leq } \text{cost}^\circ$, $h \text{ cost} = F(\text{cost}) k$, $F(\text{leq}) k \sqsubseteq k \text{ leq}$	monotonicity when the cost is itself a fold with a step k monotonic on leq	
$F(R \cap (H^\circ H)) h \sqsubseteq h R$ Proposition 9.3, $R \triangleq \text{cost} \text{ leq } \text{cost}^\circ$, $h \text{ cost} = F(\langle \text{cost}, H^\circ \rangle) k$, $F(\text{leq} \times \mathbb{1}) k \sqsubseteq k \text{ leq}$, H° simple	monotonicity in context : k may also read the input the part was built from	
$Q \triangleq F(U, V)$ Proposition 9.4, U, V preorders, $F(U, R) h \sqsubseteq h R$, $V^\circ H \sqsubseteq H R^\circ$	both conditions of Theorem 9.2 at once, split along the two arguments of a bifunctor	

§20.2. The string edit problem

$\text{Op} ::= \text{cpy Char} \mid \text{del Char} \mid \text{ins Char}$, $F(A, B) = 1 + (A \times B)$, $\alpha \triangleq [\text{nil}, \text{cons}]$.

(20.2a)

$\text{edit} \triangleq ([\text{base}, \text{step}]) : \text{list Op} \rightarrow \text{list Char} \times \text{list Char}$, base returning $([], [])$, $\text{step}(\text{cpy } a, (x, y)) = ([a] \# x, [a] \# y)$, $\text{step}(\text{del } a, (x, y)) = ([a] \# x, y)$, $\text{step}(\text{ins } a, (x, y)) = (x, [a] \# y)$.

$\text{length} \triangleq ([\text{zero}, \text{outr succ}])$, $R \triangleq \text{length} \text{ leq } \text{length}^\circ$, $U \triangleq \tau$, $V \triangleq \text{suffix} \times \text{suffix}$, $Q \triangleq \mathbb{1} + (U \times V)$, empty the coreflexive on (x, y) with both lists empty.

unstep implements $\Lambda(\text{step}^\circ) \text{thin}(U \times V)$: $\text{unstep}([a] \# x, []) = [(\text{del } a, (x, []))]$, $\text{unstep}([], [b] \# y) = [(\text{ins } b, ([], y))]$, $\text{unstep}([a] \# x, [b] \# y) = (a = b \rightarrow [(\text{cpy } a, (x, y))], [(\text{del } a, (x, [b] \# y))], (\text{ins } b, ([a] \# x, y)))$.

the law	what it says	(20.2b)
$\Lambda(\text{edit}^\circ) \min R$	the specification: a shortest edit sequence from which both strings can be reconstituted	
$F(R) \alpha \sqsubseteq \alpha R$ Proposition 9.2 at <code>length</code> , <code>succ</code> monotonic on <code>leq</code>	<code>cons</code> is monotonic on <code>R</code> , so Theorem 9.1 already applies	
$Q^\circ F(1, \text{edit}^\circ) \alpha \sqsubseteq F(1, \text{edit}^\circ) \alpha R^\circ$	the thinning condition sought, over $F(0p, \text{list Char} \times \text{list Char})$	
$F(T, R) \alpha \sqsubseteq \alpha R$ left as an exercise	$U \triangleq T$ costs nothing: any two operations may be compared	
<code>edit (1 × suffix) ⊆ R edit</code> <code>edit (suffix × 1) ⊆ R edit</code>	the other half of Proposition 9.4 for $V \triangleq \text{suffix} \times \text{suffix}$	
<code>edit (1 × tail) ⊆ R edit</code> <code>edit (tail × 1) ⊆ R edit</code> <code>suffix = tail*</code> , and $B A \sqsubseteq C B \implies B A^* \sqsubseteq C^* B$	one step is enough: drop the operation that produced the head, or weaken its <code>cpy</code> to a <code>del</code> — never lengthening the sequence	
$(\mu X : \Lambda([\text{base}, \text{step}]^\circ) \text{thin } Q P([\text{nil}, (1 \times X) \text{cons}]) \min R) \sqsubseteq \Lambda(\text{edit}^\circ) \min R$ Theorem 9.2	a copy, when available, beats a delete or an insert	
$X = (\text{empty} \rightarrow \text{nil}, \Lambda(\text{step}^\circ) \text{thin}(U \times V) P((1 \times X) \text{cons}) \min R)$ Proposition 9.1	<code>base</code> and <code>step</code> have disjoint ranges	
<code>mle = (empty → nil, unstep list((1 × mle) cons) minlist R)</code>	the program: at most two decompositions survive, but the same subproblem is solved many times, so the running time is exponential	
<code>column x y = [mle (u, y) u ← tails x]</code> <code>column x = ([fstcol x, nextcol x])</code> , <code>fstcol = list(del) tails</code>	the tabulation: <code>mle (x, y)</code> needs <code>mle (u, v)</code> for every tail <code>u</code> of <code>x</code> and <code>v</code> of <code>y</code> , so the columns are built right to left	
<code>column x ([b] # y) = nextcol x (b, column x y)</code> <code>nextcol x (b, us)</code> = <code>([base (b, last us), step b]) xus</code> <code>xus = zip (x, zip (init us, tail us))</code>	each column is a fold built bottom to top, over <code>x</code> zipped with the adjacent pairs of the column to its right	
<code>base (b, u) = [[ins b] # u]</code> <code>step b ((a, (u, v)), ws) =</code> <code>(a = b → [[cpy a] # v] # ws,</code> <code>[bmin R ([del a] # w, [ins b] # u)] # ws)</code> <code>w = head ws</code> ; these <code>base, step</code> are not <code>edit</code> 's	an entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths	

§20.3. Optimal bracketing

<pre> tree A ::= tip A bin (tree A, tree A), F X = A + X², so F(R) = 1 + R²; h ≜ [tip, bin], flatten ≜ ([wrap, cat]) : tree A → list⁺ A, H = flatten[°]. (cost, size) ≜ ([opt, opb]), opt ≜ (zero, st), opb ((cx, sx), (cy, sy)) = (cb (sx, sy) + cx + cy, sb (sx, sy)). sb associative, so size = flatten sz for a map sz; R ≜ cost leq cost[°], g ≜ [zero, (1 × sz)² opb outl], single the coreflexive on singleton lists. splits ≜ (inits⁺, tails⁺) zip, an implementation of $\Lambda(\text{cat}^\circ)$; array ≜ inits list(row), row ≜ tails list(mct), col ≜ inits list(mct). mix ≜ zip list(bin) minlist R, next ≜ (outl, mix) snoc, process ≜ ((tip wrap) × 1) loop(next). </pre>	(20.3a)
---	---------

the law	what it says	(20.3b)
$\Lambda(\text{flatten}^\circ) \min R$	the specification: a least-cost bracketing of $a_1 \otimes \dots \otimes a_n$, the tree whose flattening is the given list	
$[\text{tip}, \text{bin}] \text{ cost} = (1 + (\text{cost}, \text{flatten})^2) g \quad (9.5)$	the cost of a node reads only the cost and the flattening of its two subtrees	
$(1 + (\text{leq} \times 1)^2) g \sqsubseteq g \text{ leq} \quad (9.6)$	g is monotonic on leq in its two cost arguments	
$F(R \cap (\text{flatten} \text{ flatten}^\circ)) h \sqsubseteq h R$ Proposition 9.3, $H^\circ = \text{flatten}$ a map	monotonicity in context: only trees with the same flattening are compared	
$(\mu X : \Lambda([\text{wrap}, \text{cat}]^\circ) P([\text{tip}, (X \times X) \text{ bin}]) \min R) \sqsubseteq \Lambda(\text{flatten}^\circ) \min R$ Theorem 9.1	split the list in every way, bracket both halves, join	
$X = (\text{single} \rightarrow \text{wrap}^\circ \text{ tip}, \Lambda(\text{cat}^\circ) P((X \times X) \text{ bin}) \min R)$ Proposition 9.1	wrap and cat have disjoint ranges	
$\text{mct} = (\text{single} \rightarrow \text{head tip}, \text{splits list}((\text{mct} \times \text{mct}) \text{ bin}) \text{ minlist } R)$	the program; exponential, since the segments of one list overlap	
$\text{mct} = (\text{single} \rightarrow \text{head tip}, (\text{init col}, \text{tail row}) \text{ mix}) \quad (9.7)$	the tabulation: $\text{mct } y$ is needed for every non-empty segment y , so the values are held as an array of rows	
$\text{col} = (\text{single} \rightarrow \text{head tip wrap}, (\text{init col}, \text{tail row}) \text{ next}) \quad (9.8)$ $\text{cons col} = (1 \times \text{array}) \text{ process} \quad (9.9)$ $\text{row} = (\text{single} \rightarrow \text{head tip wrap}, (\text{mct}, \text{tail row}) \text{ cons}) \quad (9.10)$	a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop	
$\text{array} = ([\text{fstcol}, \text{addcol}]), \text{fstcol} \triangleq \text{tip wrap wrap}$ $\text{addcol} \triangleq (\text{outl tip wrap}, \text{step}) \text{ cons}$ $\text{step} \triangleq (\text{process tail}, \text{outr}) \text{ zip list}(\text{cons})$	the program: one fold building the array column by column, cubic in the length of the input	

§20.4. Data compression

<pre>list A ::= nil snoc (list A, A), list⁺ A ::= wrap A snoc (list⁺ A, A), String = list Char, Code ::= sym Char ptr (String, String⁺), F(Code, String) = 1 + (String × Code), α ≜ [nil, snoc]. decode ≜ ([nil, extend]) : list Code → String, extend (x, sym a) = x # [a], extend (x, ptr (y, z)) = x # z when y # z is a proper prefix of x # z; H = decode[°]. size ≜ ([zero, distr [1 × c, 1 × p] plus]) with c, p the constant costs of a symbol and a pointer; R ≜ size leq size[°], Q ≜ F(T + T, prefix) = 1 + (prefix × (T + T)), the two T on symbols and on pointers. lrt w = min(prefix × (T + T)) {(x, (y, z)) x # z = w, y # z a proper prefix of w}, the longest repeated tail; reduce (w # [a]) = (z ≠ [] → [(w, sym a), (x, ptr (y, z))], [(w, sym a)]) with (x, (y, z)) = lrt (w # [a]).</pre>	(20.4a)
--	---------

the law	what it says
$\Lambda(\text{decode}^\circ) \min R$	the specification: a smallest code sequence decoding to the given string
$F(R) \alpha \sqsubseteq \alpha R \quad (\tau + \tau) [c, p] = [c, p]$	monotonicity is routine, the two costs being constants
$F(\tau + \tau, R) \alpha \sqsubseteq \alpha R$ $\text{decode prefix} \sqsubseteq R \text{ decode}$ Proposition 9.4 at $Q \triangleq F(\tau + \tau, \text{prefix})$	the thinning condition split in two: pointers are ordered only by how much input they consume, symbols and pointers not at all
$\text{decode init} \sqsubseteq R \text{ decode}$	enough for the second: drop the last character of the output by shortening or removing the last code element, never raising the cost
$(\mu X : \Lambda([\text{nil}, \text{extend}]^\circ) \text{thin } Q P([\text{nil}, (X \times 1) \text{snoc}]) \min R) \sqsubseteq \Lambda(\text{decode}^\circ) \min R$ Theorem 9.2	between a symbol and a pointer nothing can be decided in advance; between two pointers the longer match wins
$X = (\text{null} \rightarrow \text{nil}, \Lambda(\text{extend}^\circ) \text{thin}(\text{prefix} \times (\tau + \tau)) P((X \times 1) \text{snoc}) \min R)$ Proposition 9.1	nil and extend have disjoint ranges
$\Lambda(\text{extend}^\circ) (w \# [a]) = \{(w, \text{sym } a)\} \cup \{(x, \text{ptr } (y, z)) \mid x \# z = w \# [a], y \# z \text{ a proper prefix of } w\}$	the decompositions of one string: take the last character as a symbol, or end with a pointer
reduce implements $\Lambda(\text{extend}^\circ) \text{thin}(\text{prefix} \times (\tau + \tau))$	thinning leaves at most two: the symbol, and the pointer of lrt
$\text{encode} = (\text{null} \rightarrow \text{nil}, \text{reduce list}((\text{encode} \times 1) \text{snoc}) \text{minlist } R)$	the program, again exponential; the book gives no tabulation for it

§21. Greedy Algorithms

§21.1. Theory

h, T, R, H, M as in (20.1a); additionally Q a **connected** preorder on the sets $\Lambda(T^\circ)$ returns, so that $\Lambda(T^\circ) \min Q$ is entire.

(21.1a)

the law	what it says
$(\mu X : \Lambda(T^\circ) \min Q F(X) h) \sqsubseteq M$ Theorem 10.1, the hypotheses of Theorem 9.2	greedy : one decomposition is kept at every step, so P and Λ disappear from the recursion
$\Lambda([V_1, V_2]^\circ) \min(Q_1 + Q_2) [U_1, U_2]$ $= (\text{Ran } V_1 \rightarrow W_1, W_2)$ $W_i \triangleq \Lambda(V_i^\circ) \min(Q_i) U_i$ Proposition 10.1, $V_2 V_1^\circ = \perp$	Proposition 9.1 with $\min$ for thin
$Q \triangleq F(U, V)$ Proposition 9.4	still the way to get both conditions, but $\Lambda(T^\circ) \min Q$ must now be entire as well
Theorem 7.2, (18.2d)	that greedy theorem chooses among the results of one relational step of a catamorphism; Theorem 10.1 chooses among the decompositions of the input, for an arbitrary T rather than an initial algebra, at the cost of h being a map

(21.1b)

§21.2. The detab-entab problem

$\text{detab} \triangleq \langle [\text{nil}, \text{expand}] \rangle : \text{String} \rightarrow \text{String}$ over snoc-lists, $\alpha \triangleq [\text{nil}, \text{snoc}]$, $H = \text{detab}^\circ$; $\text{expand}(x, a) = (a = \text{TB} \rightarrow \text{fill } x, x \# [a]), \text{fill } x = x \# \text{blanks } (n - (\text{col } x) \bmod n)$.

(21.2a)

$\text{col} \triangleq \langle [\text{zero}, \text{count}] \rangle$, $\text{count}(c, a) = (a = \text{NL} \rightarrow 0, c + 1)$; TB the tab, BL the blank, NL the newline, tab stops every n columns.

$R \triangleq \text{length} \text{ leq } \text{length}^\circ$, U the preorder with $a U b \iff b = \text{TB} \vee a = b$, $V \triangleq \text{prefix } n (\text{fill } \text{fill}^\circ)$, $Q \triangleq 1 + (V \times U)$.

$\text{unfill } x$ the shortest prefix of x with $\text{fill} (\text{unfill } x) = \text{fill } x$; tbc the trailing blank count, $\text{triple} \triangleq \langle \text{unfill } \text{entab}, \langle \text{tbc}, \text{col} \rangle \rangle$.

the law	what it says	(21.2b)
$\Lambda(\text{detab}^\circ) \min R$	the specification: a shortest input detab expands to the given output — detab entab = 1 and nothing shorter does	
$F(T, R) \alpha \sqsubseteq \alpha R$ left as an exercise	U may be any preorder on characters; $a U b \iff b = TB \vee a = b$ puts TB on top, so min prefers a tab to a blank	
$\text{detab prefix} \sqsubseteq R \text{ detab}$ FALSE	at $n = 8$, detab $[a,b,c,d,e,TB] = [a,b,c,d,e,BL,BL,BL]$, whose prefix $[a,b,c,d,e,BL,BL]$ is longer than any input giving it	
$\text{detab } V \sqsubseteq R \text{ detab } V \triangleq \text{prefix } n \text{ (fill fill}^\circ)$	true once only the prefixes that do not cross a tab stop are allowed	
$\text{nil } V = \text{nil}$ $\text{fill } V = \text{fill}$ $\text{snoc } V \sqsubseteq \text{snoc } U \text{ (outl } V)$ left as exercises	the three properties of V the derivation rests on; the second is what fill fill [°] was added for	
$\text{expand } V \sqsubseteq \text{expand } U \text{ (outl } V)$	the claim they add up to: shortening the output either leaves the last step alone or discards it	
$\text{detab } V \sqsubseteq \text{detab } U \text{ (init detab } V)$ <i>init</i> inductive, so the greatest solution is the unique one	hence detab $V \sqsubseteq \text{prefix detab}$, and prefix $\sqsubseteq R$ finishes Proposition 9.4	
$(\mu X : \Lambda([\text{nil}, \text{expand}]^\circ) \min Q (1 + (X \times 1)) [\text{nil}, \text{snoc}]) \sqsubseteq \Lambda(\text{detab}^\circ) \min R$ Theorem 10.1	greedy: one character of input is decided at each step	
$X = (\text{null} \rightarrow \text{nil}, \Lambda(\text{expand}^\circ) \min(V \times U) (X \times 1) \text{snoc})$ Proposition 10.1	nil and expand have disjoint ranges	
$\Lambda(\text{expand}^\circ) (x \# [a]) = \{(y, TB) \mid \text{fill } y = x \# [a]\} \cup \{(x, a)\}$ the first set is non-empty iff $a = BL$ and $\text{col } (x \# [a]) \bmod n = 0$	the last output character came from a tab only if it is a blank landing exactly on a tab stop	
$\Lambda(\text{expand}^\circ) \min(V \times U) (x \# [a]) = (a = BL \wedge \text{col } (x \# [a]) \bmod n = 0 \rightarrow (\text{unfill } x, TB), (x, a))$	the greedy step: emit a tab whenever a tab is legal, consuming all the blanks back to the previous tab stop	
$\text{entab } x = \text{entab } (\text{unfill } x) \# \text{blanks } (\text{tbc } x)$ (10.1)	what makes triple a snoc -list catamorphism: the output splits at the last tab stop	
triple = $\langle [\text{base}, \text{op}] \rangle$ and entab = triple assocl outl $(1 \times \text{blanks}) \text{ cat}$	the program: one pass carrying the column and the count of pending blanks	
base returns $([], (0, 0))$ op $((x, (t, c)), a) =$ $(a = BL \wedge (c + 1) \bmod n \neq 0 \rightarrow (x, (t + 1, c + 1)),$ $a = BL \rightarrow (x \# [TB], (0, c + 1)),$ $a = NL \rightarrow (x \# \text{blanks } t \# [NL], (0, 0)),$ $(x \# \text{blanks } t \# [a], (0, c + 1)))$	hold a blank back, cash the held blanks in for a tab at a tab stop, flush them at a newline, flush them before anything else	

§21.3. The minimum tardiness problem

$F X = 1 + (X \times \text{Job}), \alpha \triangleq [\text{nil}, \text{snoc}]$ on schedules, $\beta \triangleq [\text{nil}, \text{snag}]$ on bags, snag putting a job into a bag; bagify $\triangleq \langle \beta \rangle : \text{list Job} \rightarrow \text{Bag Job}, H = \text{bagify}^\circ$. ct, dt, wt : $\text{Job} \rightarrow \text{Real}$ the completion, due and weighting quantities of a job; penalty $(x, j) = (\text{sum } (\text{list}(\text{ct } x) + \text{ct } j - \text{dt } j) \times \text{wt } j)$. cost $\triangleq \Lambda(\text{prefix}) P(\alpha^\circ [\text{zero}, \text{penalty}]) \max \text{leq}$, cost $[] = 0$, cost $(x \# [j]) = \text{bmax } (\text{cost } x, \text{penalty } (x, j))$, $R \triangleq \text{cost leq cost}^\circ$. perm $\triangleq \text{bagify bagify}^\circ = \langle [\text{nil}, \text{add}] \rangle$, add $(x, j) = y \# [j] \# z$ for some $x = y \# z$. $k \triangleq [\text{zero}, \text{assocr } (1 \times ((\text{bagify}^\circ \times 1) \text{penalty})) \text{bmax}]$, $f \triangleq [\text{zero}, (\text{bagify}^\circ \times 1) \text{penalty}]$, $Q \triangleq f \text{leq } f^\circ$.	(21.3a)
--	---------

the law	what it says	(21.3b)
$\Lambda(\text{bagify}^\circ) \min R$	the specification: an ordering of the given bag of jobs with least maximum penalty	
$F(R \cap (\text{bagify } \text{bagify}^\circ)) \alpha \sqsubseteq \alpha R \quad (10.2)$	monotonicity in context — only schedules of one bag are compared	
$(Q^\circ \cap (\beta \beta^\circ)) F(\text{bagify}^\circ) \alpha \sqsubseteq F(\text{bagify}^\circ) \alpha R^\circ \quad (10.3)$	the greedy condition, also in context	
$\text{cost } (x \# [j]) = \text{bmax } (\text{cost } x, \text{penalty } (\text{perm } x, j))$ $\alpha \text{ cost} = F((\text{cost}, \text{bagify})) k, F(\text{leq } \times 1) k \sqsubseteq k \text{ leq}$ Proposition 9.3	(10.2): penalty sums the completion times of x , and a sum does not see the order	
$\alpha \text{ cost} = \langle g, h \rangle \text{bmax} \quad (10.4)$ $g \triangleq [\text{zero}, \text{penalty}] \quad (10.5)$ $h \triangleq [\text{zero}, \text{outl cost}] \quad (10.6)$	the cost of a schedule split into the penalty of its last job and the cost of the rest	
$\text{add} \sqsubseteq \text{outl } R^\circ \quad (10.7)$ $\beta \text{ bagify}^\circ = F(\text{bagify}^\circ) [\text{nil}, \text{add}] \quad (10.8)$	adding a job never lowers the cost; and bagify $^\circ$ is itself a catamorphism on bags. Together: $\beta \text{ bagify}^\circ \sqsubseteq F(\text{bagify}^\circ) h \text{ geq cost}^\circ$	
$F(\text{bagify}) Q^\circ F(\text{bagify}^\circ) = g \text{ geq } g^\circ$, met by $Q \triangleq f \text{ leq } f^\circ, f \triangleq [\text{zero}, (\text{bagify}^\circ \times 1) \text{ penalty}]$	the specification Q has to satisfy, and the choice that meets it: a Q -minimum is a job of least penalty	
no greedy catamorphism exists	a greedy snoc-list catamorphism would also solve every prefix of the input, and the best schedule of a prefix need not extend to a best schedule of the whole	
$X = (\text{null} \rightarrow \text{nil}, \Lambda(\text{snag}^\circ) \min Q (X \times 1) \text{ snoc})$ Theorem 10.1, Proposition 10.1	nil and snag have disjoint ranges	
$\text{schedule} = (\text{null} \rightarrow \text{nil}, \text{pick } (\text{schedule} \times 1) \text{ snoc})$ $\text{pick} \sqsubseteq \Lambda(\text{snag}^\circ) \min Q$, a partial function	the program: repeatedly remove a job of least penalty and put it last; quadratic in the number of jobs	

§21.4. The TeX problem

$\text{intern} \triangleq \text{val round} : \text{Decimal} \rightarrow [0, 2^{16}), \text{val} \triangleq ()[\text{zero}, \text{shift}], \text{shift } (d, r) = (d + r)/10, \text{round } r \text{ rounds } 2^{16} r \text{ to the nearest integer: } \text{round } r = n \iff 2n - 1 < 2^{17} r < 2n + 1.$ $\text{interval } n = ((2n - 1)/2^{17}, (2n + 1)/2^{17}), r \text{ inrange } (a, b) \iff a < r < b, \text{round}^\circ = \text{interval inrange}, R \triangleq \text{length leq length}^\circ.$ Interval the pairs (a, b) with $0 < b < 1$ and $a < b$ (10.9) $[\text{arb}, \text{step}] : 1 + (\text{Digit} \times \text{Interval}) \rightarrow \text{Interval}, \text{step } (d, (a, b)) = ((d + a)/10, (d + b)/10).$ $F X = 1 + (\text{Digit} \times X), \alpha \triangleq [\text{nil}, \text{cons}], h \triangleq ()[\text{arb}, \text{step}] = H^\circ, ! : \text{Digit} \times \text{Interval} \rightarrow 1, Q \triangleq (\text{!}_r^\circ ! \text{!}_l) \cup 1, w \triangleq 2^{17}.$	(21.4a)
--	---------

the law	what it says
$\Lambda(\text{intern}^\circ) \min R = \text{interval } \Lambda(\text{inrange } \text{val}^\circ) \min R$	the specification: a shortest decimal whose internal representation is the given multiple of 2^{-16} . round° is not a map, but interval is, so it comes out of the Λ
$\text{val } \text{inrange}^\circ = ([\text{arb}, \text{step}])$ fusion; $\text{zero } \text{inrange}^\circ = \text{arb}, \text{shift } \text{inrange}^\circ = (1 \times \text{inrange}^\circ) \text{ step}$	the converse of val , cut down to intervals, is a catamorphism on cons-lists
$F(R) \alpha \sqsubseteq \alpha R$	cons is monotonic on R — routine
$\alpha^\circ F(h) Q \sqsubseteq R \alpha^\circ F(h)$	the greedy condition of Theorem 10.1, converted
$\Lambda(\text{arb}^\circ) (a, b) = (a < 0 \rightarrow \{*\}, \{\})$ $\Lambda(\text{step}^\circ) (a, b) = \{(d, (10a - d, 10b - d)) \mid 0 < 10b - d < 1\}$	the decompositions of an interval: stop, or take one more digit
$0 < 10b - d_1 < 1 \text{ and } 0 < 10b - d_2 < 1 \text{ imply } d_1 = d_2$	step° is in fact a map, d the digit with $0 < 10b - d < 1$, so $\Lambda([\text{arb}, \text{step}]^\circ)$ returns at most two elements and Q need only choose between them
$Q \triangleq (\text{! } r^\circ \text{ ! } \text{!}) \cup \text{!}$, and $\text{! } \text{nil} \sqsubseteq \text{cons } R$ from $\text{! } \text{nil } \text{length} \sqsubseteq \text{cons } \text{length } \text{leq}$	stop whenever stopping is legal: the empty decimal is shorter than any other
$(\mu X : \Lambda([\text{arb}, \text{step}]^\circ) \min Q F(X) \alpha) \sqsubseteq \Lambda(\text{intern}^\circ) \min R$ Theorem 10.1	greedy: emit the one digit the interval allows, until the interval contains zero
$\text{extern} = \text{interval } f$ $f (a, b) = (a < 0 \rightarrow [], [d] \# f (10a - d, 10b - d))$	the program, with d the digit above
$\text{extern } n = f (2n - 1, 2n + 1)$ $f (p, q) = (p \leq 0 \rightarrow [], [d] \# f (10p - w d, 10q - w d))$ $d = (10q) \text{ div } w, w = 2^{17}$	the same in integer arithmetic only, as chapter 3 required of intern : every interval reached is $(p/w, q/w)$